

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

**Génération de trajectoire robotique sans collision dans un environnement
dynamique utilisant des modèles de Flow Matching**

FRÉDÉRICK LANTHIER

Département de génie mécanique

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Génie mécanique

Septembre 2026

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

Ce mémoire intitulé :

**Génération de trajectoire robotique sans collision dans un environnement
dynamique utilisant des modèles de Flow Matching**

présenté par **Frédéric LANTHIER**

en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente

Sofiane ACHICHE, membre et directeur de recherche

Jérôme LE NY, membre et codirectrice de recherche

Jean TREMBLAY, membre

Joseph BROWN, membre externe

DÉDICACE

*À ma blonde que j'aime, mes parents,
et mes amis des labs...*

REMERCIEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE DES MATIÈRES

DÉDICACE	iv
REMERCIEMENTS	v
RÉSUMÉ	vi
ABSTRACT	vii
LISTE DES TABLEAUX	xii
LISTE DES FIGURES	xiii
LISTE DES SIGLES ET ABRÉVIATIONS	xvi
LISTE DES ANNEXES	xvii
CHAPITRE 1 INTRODUCTION	1
CHAPITRE 2 REVUE DE LITTÉRATURE	3
2.1 Génération de trajectoires robotisées	3
2.1.1 Planification classique par échantillonnage et optimisation	3
2.1.2 Apprentissage par démonstration, clonage de comportement et renforcement	4
2.1.3 Modèles génératifs, politiques de diffusion et Flow Matching	5
2.2 Perception visuelle et mémoire spatiale	7
2.2.1 Vision-langage, segmentation zéro-shot et suivi visuel	7
2.2.2 Reconstruction RGB-D et représentation par nuages de points	8
2.2.3 Cartes persistantes face aux occlusions et pertes de détection	8
2.3 Représentations géométriques pour la collision	9
2.3.1 Champs de distance de la scène et limites computationnelles	9
2.3.2 Primitives conservatives et modèles géométriques simplifiés	10
2.3.3 SDF du robot, RDF et approximation par polynômes de Bernstein	11
2.4 Sécurité réactive et Control Barrier Functions	12
2.4.1 Champs de potentiel et méthodes réactives classiques	12
2.4.2 Fonctions barrières de contrôle et invariance avant	12
2.4.3 Limites des CBF sous perception bruitée et contrôle discret	13
2.5 Couplage entre modèles génératifs et filtres de sécurité	14

2.5.1	Contraintes intégrées à la génération	14
2.5.2	Filtres de sécurité appliqués à l'exécution	15
2.5.3	Découplage entre intention nominale et sécurité réactive	15
2.6	Synthèse critique et positionnement	16
2.6.1	Lacunes identifiées dans la littérature	16
2.6.2	Positionnement de l'architecture proposée	16
2.6.3	Hypothèse de recherche	16
CHAPITRE 3	MOTIVATION DE RECHERCHE ET OBJECTIFS	19
3.1	Contexte et problématique	19
3.2	Objectifs de recherche	19
3.3	Contexte et problématique	19
3.4	Portée et exclusions de la recherche	19
CHAPITRE 4	DESIGN DU SYSTÈME	20
4.1	Plateforme Expérimentale Et Architecture Globale	20
4.1.1	Scène robotique, capteur RGB-D, robot manipulateur	20
4.1.2	Architecture modulaire du système	20
4.2	Perception Sémantique et Reconstruction 3D	21
4.2.1	Extraction Sémantique	21
4.2.2	Suivi temporel de la cible	21
4.2.3	Projection RGB-D vers l'espace 3D	22
4.2.4	Séparation cible, obstacles et outil visible	22
4.2.5	Nettoyage spatial du nuage d'obstacles	22
4.3	Cartes Persistantes de la scène	22
4.3.1	Motivation : robustesse aux pertes de détection et occlusion	22
4.3.2	Génération des cartes persistantes	22
4.3.3	Mise à jour des cartes persistantes	22
4.4	Acquisition des données et représentation des démonstrations	22
4.4.1	Protocol de collecte des démonstrations	22
4.4.2	Prétraitement des données	26
4.5	Génération de trajectoire par Flow Matching	29
4.5.1	Formulation mathématique du Flow Matching conditionnel	29
4.5.2	Architecture du modèle	30
4.5.3	Entraînement du modèle	32
4.6	Conversion et exécution de la trajectoire nominale	32
4.6.1	Conversion outil - TCP	32

4.6.2	Cinématique inverse pour obtenir la trajectoire articulaire nominale .	32
4.6.3	Réajustement temporel de la trajectoire articulaire	33
4.7	Filtre de sécurité réactif SDF-CBF	34
4.7.1	Modèle de distance du robot appris hors ligne	35
4.7.2	Fonction barrière	37
4.7.3	Gradient analytique de la contrainte	38
4.7.4	Contrainte de sécurité et vitesse nominale	41
4.7.5	De la vitesse nominale à la commande sécurisée	42
CHAPITRE 5	RÉSULTATS ET DISCUSSION	47
5.1	Protocol expérimental	48
5.1.1	Description des scénarios testés	48
5.1.2	Métriques utilisées	48
5.2	validation du modèle géométrique de sécurité	48
5.2.1	Précision du SDF du robot	49
5.2.2	Temps d'évaluation du SDF et du gradient	51
5.2.3	Visualisation des points critiques sélectionnés	51
5.3	Qualité de la génération nominale	53
5.3.1	Trajectoire générée sans obstacle critique	53
5.3.2	Cohérence de la trajectoire avec la cible	53
5.4	Sécurité et évitement d'obstacle	53
5.4.1	Comparaison des trajectoires avec et sans SDF-CBF	53
5.4.2	Évolution de $h(q)$ et de la contrainte de sécurité	53
5.4.3	Déviations entre trajectoire nominale et trajectoire sécurisée	58
5.5	Analyse des vitesses et de la fluidité	58
5.5.1	Décomposition des vitesses articulaires	58
5.5.2	Action du filtre de sécurité sur la vitesse commandée	58
5.5.3	Effet du rééchantillonnage du planificateur sur la vitesse exécutée . .	60
5.5.4	Effet du filtrage sur la commande sécurisée	61
5.5.5	Analyse de l'interface de commande	61
5.6	Performance et temps de calcul	63
5.7	Performances globales et limites	63
5.7.1	Taux de succès des scénarios	63
5.7.2	Cas d'échec	63
5.8	Discussion	63
CHAPITRE 6	CONCLUSION	64

6.1 Synthèse des travaux / Summary of Works	64
6.2 Limitations de la solution proposée / Limitations	64
6.3 Améliorations futures / Future Research	64
RÉFÉRENCES	65
ANNEXES	72

LISTE DES TABLEAUX

Tableau 2.1	Tableau comparatif des frameworks de planification et de sécurité de l'état de l'art.	17
Tableau 4.1	Temps d'évaluation conjointe des barrières h_l et de la Jacobienne de contrainte complète ($L \times 7$) pour 100 points obstacles, en fonction du nombre L de liens protégés inclus : gradient analytique en forme close contre différentiation automatique (moyenne $\pm$ écart-type en ms, 100 répétitions), avec l'erreur absolue maximale entre les deux Jacobiennes.	41
Tableau 4.2	Paramètres du filtre de sécurité SDF-CBF.	46
Tableau 5.1	Statistiques descriptives par étape (stage)	63

LISTE DES FIGURES

Figure 4.1	Architecture globale du système asynchrone.	20
Figure 4.2	Exemple de masque obtenu par le modèle de segmentation SAM3 pour le prompt <i>Pink Block</i>	21
Figure 4.3	Protocole d'échantillonnage des cibles, vu de dessus dans le repère de base du robot $\mathcal{F}_b$. Les assiettes (26 cm de diamètre) se chevauchant, seules trois des neuf positions de la grille 3×3 sont illustrées (supérieur droit, centre, inférieur gauche), le centre de chacune étant marqué par une croix noire. Autour de chaque centre, les 5 points indiquent les positions de cible en quinconce.	24
Figure 4.4	Architecture du système d'acquisition de données	25
Figure 4.5	Directory structure of the PickPlace dataset execution tests.	26
Figure 4.6	Représentation schématique des frames des différents repères utilisés	28
Figure 4.7	Structure du dossier du jeu de données des trajectoires prétraités	29
Figure 4.8	Architecture du module ConditionalResidualBlock1D avec un tracé orthogonal à angle droit unique vers les entrées latérales.	30
Figure 4.9	Architecture globale finale du ConditionalUnet1D au style épuré, corrigée de tout dépassement de tracé.	31
Figure 5.1	Précision de la distance du Signed Distance Field (SDF) de Bernstein. (a) Profil de distance le long des normales à la surface du robot ; (b) distribution de l'erreur signée par rapport à la distance exacte calculée sur les maillages.	49
Figure 5.2	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	50
Figure 5.3	Trajectoire multimodale générée par <i>Flow Matching</i>	50
Figure 5.4	Temps de calcul de la distance et du gradient : modèle de Bernstein (Graphics Processing Unit (GPU)) contre vérité terrain sur maillages (Central Processing Unit (CPU)), en fonction du nombre de points de requête.	51
Figure 5.5	Sélection des points les plus proches pour contraindre le CBF	52
Figure 5.6	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	53

Figure 5.7	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	54
Figure 5.8	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	54
Figure 5.9	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	55
Figure 5.10	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	55
Figure 5.11	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	56
Figure 5.12	Évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} pour une tâche en présence d'obstacles ; les bandes vertes indiquent les cycles où la projection corrige la vitesse, les bandes jaunes ceux où seule la réparation intervient.	57
Figure 5.13	Contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après réparation). Les valeurs négatives de la courbe nominale correspondent aux violations qu'aurait commises la trajectoire générée sans filtre.	57
Figure 5.14	Décomposition des vitesses articulaires. (a) Construction de la vitesse nominale : feedforward retimé à norme constante $\dot{\mathbf{q}}_{\text{ff}}$, correction de suivi $\dot{\mathbf{q}}_{\text{fb}}$ et vitesse nominale filtrée $\dot{\mathbf{q}}_{\text{base}}$. (b) Action du filtre de sécurité : corrections de projection $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ et de réparation $\Delta\dot{\mathbf{q}}_{\text{rep}}$ (reconstituée par rejeu du filtre de sortie), et vitesse commandée $\dot{\mathbf{q}}_{\text{final}}$. Courbes lissées sur 0,25 s.	59
Figure 5.15	Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.	60

Figure 5.16	Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.	61
Figure 5.17	Norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie ($\tau_{\text{safe}} = 0,2$ s).	62

LISTE DES SIGLES ET ABRÉVIATIONS

API	Application Programming Interface
ARA	Assistive Robotic Arm
BC	Behavior Cloning
CBF	Constraint Barrier Function
CFM	Constraint Flow Matching
CPU	Central Processing Unit
DDL	Degré de liberté
FM	Flow Matching
GPU	Graphics Processing Unit
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model
QP	Quadratic Problem
RAF	Robotic Assisted Feeding
RDF	Robot Distance Field
RGB-D	Red-Green-Blue + Depth
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
ROS	Robot Operating System
SAM	Segment Anything Model
SDF	Signed Distance Field
SO	Specific objective
VLM	Vision Language Model
YAML	YAML Ain't Markup Language

LISTE DES ANNEXES

Annexe A	Démo	72
Annexe B	Encore une annexe / Another Appendix	73
Annexe C	Une dernière annexe / The Last Appendix	74

CHAPITRE 1 INTRODUCTION

Présenter la problématique de recherche, comme quoi l'utilisation des modèles d'IA générative pour des tâches robotiques réelles ne sont pas couramment utiliser puisqu'ils nécessite le positionnement d'une caméra externe, parce que c'est dans des environnements dynamiques et que si on veut qu'ils soient réactifs aux collisions, il faut entraîner le modèle sur plusieurs centaines de données au lieu d'une quarantaine pour une tâche afin de lui faire faire apprendre l'évitement de collision. cela complexifie alors parler aussi des robots collaboratifs qui permettent de réaliser des tâches, mais que ces tâches sont pour la plupart hardcoder et ne réagissent pas aux stimuli environnants, ce qui peut générer des trajectoires dangereuses.

Parler de la motivation de la recherche. Pour moi, serait que la génération de trajectoire par intelligence artificielle, pour ne pas accumuler des centaines de données pour faire de l'évitement de collision doit avoir un système d'évitement de collision sur la trajectoire réalisée. Ce système peut se trouver lors de l'inférence du modèle ou bien après l'inférence du modèle. Lors de l'inférence du modèle, c'est trop long pour des environnements dynamiques et après il faut considérer l'environnement, ce qui nécessite l'entraînement d'un réseau que pour l'environnement. De plus, les systèmes actuels nécessitent une caméra externe calibrée adéquatement, ce qui n'est pas utilisable pour un usage quotidien. Ce qu'on propose c'est un système d'évitement de collision pour des environnements dynamiques pour des environnements non vues pendant l'entraînement avec une seule caméra au niveau de l'effecteur du robot.

Faire un paragraphe par rapport à la structure du mémoire. Celle de Félix c'était : Chapter 2 reviews the literature on robot-assisted feeding, highlighting current challenges in perception, task planning, and manipulation. Chapter 3 defines the specific scope and research objectives of this project. Chapter 4 details the design of the system, describing the system architecture, the integration of Vision-Language Models (VLMs) for autonomous food acquisition planning, and the implementation of the food perception and manipulation modules. Chapter 5 presents the experimental results, including an analysis of VLM-based food acquisition planning performance and a validation of the system on a physical robot. Finally, Chapter 6 summarizes the main contributions, discusses the system's limitations, and proposes directions for future research.

Donc nous cela pourrait être "Le chapitre 2 présente la revue de littérature en lien avec la génération de trajectoires robotiques en utilisant l'IA générative mettant en évidence l'exécution des trajectoires la vision par ordinateur et le suivi de cible ainsi que l'évitement de

collision. Le chapitre 3 présente la question de recherche ainsi que les sous objectifs associées pour répondre à cette hypothèse. Le chapitre 4 présente le design du système, mettant de l'avant son architecture, l'intégration des systèmes de vision par ordinateur pour détecter et suivre la cible, l'acquisition des données pour entraîner le modèle de Flow Matching, la génération des trajectoires générées par IA ainsi, la capture des obstacles, le filtre de sécurité utilisé et l'intégration du temps réel. Le chapitre 5 présente les résultats expérimentaux, la validation du modèle géométrique de sécurité, la qualité de la trajectoire nominale, l'analyse des vitesses, la sécurité et l'évitement d'obstacle, les performances computationnelles ainsi que les limites du système. "

CHAPITRE 2 REVUE DE LITTÉRATURE

Cette revue de littérature présente une analyse structurée de l'état de l'art pour la génération de trajectoires robotisées sûres en environnement dynamique et non structuré. Elle est organisée selon l'architecture en chaîne « perception, planification, action » proposée dans ce travail. Nous abordons d'abord les méthodes de génération de mouvement, en situant l'émergence du *Flow Matching* conditionnel retenu comme planificateur nominal. Ensuite, nous examinons les techniques de perception visuelle et de mémoire spatiale qui fournissent le conditionnement géométrique nécessaire. Les représentations géométriques permettant d'évaluer les collisions en temps réel sont ensuite analysées, suivies par les lois de commande sécuritaires réactives fondées sur la théorie du contrôle. Enfin, nous étudions les stratégies de couplage entre modèles génératifs et filtres de sécurité, ce qui prépare le positionnement de l'architecture découplée proposée dans ce mémoire.

2.1 Génération de trajectoires robotisées

Objectif de la section : Analyser le développement théorique et les limites connues des méthodes de génération de mouvements afin de situer le choix du *Flow Matching* conditionnel comme planificateur d'intention nominale.

2.1.1 Planification classique par échantillonnage et optimisation

La génération de mouvements sans collision a d'abord été abordée par des algorithmes déterministes raisonnant directement dans l'espace des configurations sans apprentissage préalable. Historiquement, deux grandes familles se dégagent selon qu'elles explorent cet espace par tirages aléatoires ou qu'elles raffinent une trajectoire par optimisation continue. D'une part, les méthodes par échantillonnage, à l'instar de l'algorithme *Rapidly-exploring Random Tree star* (RRT*) de Karaman et Frazzoli [1], construisent incrémentalement un arbre de configurations valides. Bien que ces planificateurs, largement implémentés au sein de la bibliothèque *Open Motion Planning Library* (OMPL) [2], garantissent une complétude probabiliste et une optimalité asymptotique, ils produisent généralement des trajectoires géométriquement rugueuses et saccadées. Ce défaut impose une étape de lissage a posteriori qui dépend fortement de la complexité géométrique de la scène [3].

D'autre part, les approches par optimisation locale adoptent une démarche inverse en raffinant une trajectoire initiale par minimisation d'une fonctionnelle de coût. Par exemple,

CHOMP (*Covariant Hamiltonian Optimization for Motion Planning*) [4] exploite le gradient d'un champ de distance signé pré-calculé pour repousser le robot hors des obstacles tout en lissant sa trajectoire. De même, TrajOpt [5] formule le problème sous forme d'une séquence d'optimisations convexes sous contraintes de non-collision, démontrant d'excellents taux de réussite sur des bras articulés complexes. Néanmoins, ces méthodes restent intrinsèquement sensibles à la trajectoire d'initialisation et tendent à converger vers des minima locaux sous-optimaux, ce qui peut conduire à des violations de contraintes si le champ de distance est bruité. Pour pallier cette dépendance à l'initialisation, les approches classiques d'amorçage (*warm-starting*) proposent d'utiliser des heuristiques géométriques ou des chemins sans collision issus d'algorithmes par échantillonnage, comme RRT-Connect [6] ou des cartes routières probabilistes (PRM) [7], pour ensemercer l'optimiseur. Ce couplage hybride échantillonnage-optimisation combine la complétude globale du premier et la régularité locale du second. L'évolution de ces stratégies classiques d'amorçage vers des représentations apprises assure une transition naturelle vers les méthodes fondées sur l'apprentissage, qui cherchent à s'affranchir de la recherche géométrique explicite en exploitant des démonstrations expertes.

2.1.2 Apprentissage par démonstration, clonage de comportement et renforcement

L'apprentissage par démonstration (*Learning from Demonstration*, LfD), dont les fondations théoriques ont été posées par Atkeson et Schaal [8] ainsi que par Schaal [9] avant d'être synthétisées par Argall et al. [10], vise à reproduire un comportement expert sans programmer explicitement la loi de commande. La formulation la plus directe, le clonage de comportement (*Behavior Cloning*, BC), dont le concept remonte aux travaux pionniers d'ALVINN de Pomerleau [11] et a été formalisé par Bain et Sammut [12], traite le problème comme une régression supervisée des actions expertes à partir des observations. Bien qu'appliqué avec succès à des tâches d'acquisition alimentaire par imitation visuelle ou de manipulation fine moderne [13–15], le BC sous perte quadratique (*Mean Squared Error*, MSE) souffre de deux limites structurelles majeures : l'erreur cumulative (*compounding errors*), où de petites déviations entraînent le robot hors de la distribution d'entraînement, et l'effondrement de la multimodalité, la moyenne de démonstrations divergentes produisant des actions aberrantes ou invalides.

Pour surmonter ces écueils, Florence et al. [16] ont introduit le clonage de comportement implicite (*Implicit BC*) représentant la politique par un modèle d'énergie. Cette formulation permet de capturer des distributions d'actions multivaluées et discontinues, surpassant la régression directe sur des tâches nécessitant une grande précision géométrique. Par ailleurs,

des travaux connexes étendent cette logique à l’imitation strictement hors ligne [17] ou exploitent des représentations auto-supervisées pour accroître la généralisation du système [18]. En parallèle, l’apprentissage par renforcement (*Reinforcement Learning*, RL) résout le problème de la multimodalité par exploration active de l’environnement [19, 20]. Toutefois, le RL se heurte à une inefficacité d’échantillonnage prohibitive et pose des risques majeurs de collision, ce qui limite son déploiement direct sur des plateformes physiques réelles. Enfin, une approche alternative consiste à encoder le mouvement sous forme de systèmes dynamiques stables par construction (tels que l’estimateur SEDS [21]) afin de garantir la convergence vers la cible, bien que cela se fasse au détriment de l’expressivité par rapport aux modèles génératifs récents.

2.1.3 Modèles génératifs, politiques de diffusion et Flow Matching

Les politiques génératives modélisent explicitement la distribution des trajectoires expertes, ce qui leur permet de représenter naturellement la multimodalité inhérente aux comportements humains [22, 23]. Récemment, la *Diffusion Policy* de Chi et al. [24], inspirée des modèles probabilistes de débruitage (*Denoising Diffusion Probabilistic Models*) [25] et de la planification générative [26, 27], a démontré des performances remarquables en manipulant des objets par un processus de débruitage stochastique itératif. De même, sa variante tridimensionnelle (*3D Diffusion Policy*) conditionne cette génération sur un nuage de points [28], renforçant la robustesse géométrique. Toutefois, le principal inconvénient des politiques de diffusion réside dans leur latence d’inférence élevée : la génération d’une trajectoire requiert de nombreuses évaluations séquentielles du réseau de neurones, limitant leur application en boucle fermée à haute fréquence. Bien que des techniques de distillation de cohérence (telles que ManiCM [29] ou d’autres approches accélérées [30, 31]) parviennent à ramener l’inférence à une seule étape, elles nécessitent des phases d’entraînement complexes et peuvent dégrader la stabilité des trajectoires générées.

Pour offrir une alternative déterministe plus directe et efficace, Lipman et al. [32] ont introduit le formalisme du *Flow Matching*. L’objectif est d’apprendre un champ de vecteurs $v_\theta(\mathbf{x}, t)$, indexé par un temps de flux $t \in [0, 1]$, qui déplace continûment un échantillon initial issu d’une distribution simple $p_0(\mathbf{x})$ (typiquement une gaussienne standard $\mathcal{N}(\mathbf{0}, \mathbf{I})$) vers la distribution cible des trajectoires expertes $p_1(\mathbf{x})$. Une trajectoire candidate $\mathbf{x}_t$ est générée en intégrant l’équation différentielle ordinaire (ODE) associée :

$$\frac{d\mathbf{x}_t}{dt} = v_\theta(\mathbf{x}_t, t). \quad (2.1)$$

Le champ de vecteurs marginal engendrant la distribution de probabilité $p_t(\mathbf{x})$ étant généralement infaisable à calculer directement, le *Conditional Flow Matching* (CFM) [33] résout ce problème en définissant un chemin de probabilité conditionnel $p_t(\mathbf{x} \mid \mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}(\mathbf{x} \mid \mu_t(\mathbf{x}_0, \mathbf{x}_1), \sigma_t^2 \mathbf{I})$ et son champ de vecteurs associé $u_t(\mathbf{x} \mid \mathbf{x}_0, \mathbf{x}_1)$ qui sont analytiquement traitables. Pour des distributions gaussiennes conditionnelles, ce champ cible s'exprime par :

$$u_t(\mathbf{x} \mid \mathbf{x}_0, \mathbf{x}_1) = \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \mu_t(\mathbf{x}_0, \mathbf{x}_1)) + \dot{\mu}_t(\mathbf{x}_0, \mathbf{x}_1), \quad (2.2)$$

où $\dot{\mu}_t$ et $\dot{\sigma}_t$ désignent les dérivées temporelles des paramètres du chemin. Le réseau de neurones v_θ est alors entraîné en minimisant la perte de régression du champ de vecteurs conditionnel :

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], \mathbf{x}_1 \sim p_1, \mathbf{x}_0 \sim p_0, \mathbf{x}_t \sim p_t(\cdot \mid \mathbf{x}_0, \mathbf{x}_1)} \left[\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t \mid \mathbf{x}_0, \mathbf{x}_1)\|^2 \right]. \quad (2.3)$$

Dans le cadre du *Rectified Flow* proposé par Liu et al. [34], le chemin est défini par interpolation linéaire simple entre le bruit et la donnée réelle, avec une variance constante :

$$\mu_t(\mathbf{x}_0, \mathbf{x}_1) = t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \quad \sigma_t = \sigma_{\min}, \quad (2.4)$$

ce qui réduit le champ cible u_t au vecteur constant reliant le point initial au point final :

$$u_t(\mathbf{x}_t \mid \mathbf{x}_0, \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0. \quad (2.5)$$

Cette formulation linéaire force les trajectoires d'intégration à être quasi rectilignes, permettant d'intégrer l'ODE en un nombre extrêmement réduit d'étapes de discrétisation sans perte de qualité. Pour adapter ce formalisme à la robotique, il suffit de conditionner le champ $v_\theta(\mathbf{x}, t \mid \mathbf{o})$ sur une observation sémantique ou géométrique $\mathbf{o}$ (comme un nuage de points), ce qui permet d'obtenir un planificateur de trajectoire réactif, comme l'illustre le framework PointFlowMatch [35].

Plus récemment, de nombreuses extensions ont cherché à accroître l'efficacité et la généralisation géométrique de ces modèles. Des approches telles que ManiFlow [36], MeanFlow [37, 38] et les politiques à flux continu [39] visent la génération de mouvements en une ou deux étapes d'inférence. D'autres travaux, à l'instar de FlowMP [40, 41], étendent le *Flow Matching* aux dynamiques du second ordre afin de générer des trajectoires respectant les contraintes d'accélération et de couple. Enfin, la politique riemannienne (*Riemannian Flow Matching Policy*, RFMP) de Braun et al. [42] transpose ce formalisme directement sur des variétés non euclidiennes telles que $SE(3)$ par interpolation géodésique, réduisant les saccades de rotation et le temps de calcul. Cette combinaison unique d'expressivité multimodale, de rapidité d'intégra-

tion déterministe et de robustesse géométrique justifie l’adoption du *Flow Matching* comme planificateur nominal d’intention au sein de notre architecture.

2.2 Perception visuelle et mémoire spatiale

Objectif de la section : Documenter les mécanismes permettant de convertir un flux de données visuelles bidimensionnelles et bruitées en un signal de conditionnement géométrique stable et persistant.

2.2.1 Vision-langage, segmentation zéro-shot et suivi visuel

La première étape du pipeline consiste à extraire de la scène l’objet pertinent pour la tâche et à l’ancrer sémantiquement. Dans cette perspective, deux grands paradigmes s’occupent d’associer perception et action : les modèles monolithiques, qui fusionnent perception et action, et les architectures modulaires, qui séparent la compréhension sémantique de la génération de mouvement. D’une part, les architectures *Vision-Language-Action* (VLA) monolithiques intègrent la perception sémantique et le contrôle dans un unique réseau de neurones entraîné de bout en bout. Des modèles tels que RT-2 de Brohan et al. [43] ou *OpenVLA* de Kim et al. [44] en sont particulièrement représentatifs. Ce dernier, fort de sept milliards de paramètres, surpasse significativement les modèles propriétaires antérieurs sur une grande variété de tâches de manipulation [45, 46]. Néanmoins, leur principale limite réside dans leur latence d’inférence élevée, peu compatible avec un contrôle réactif en temps réel. Bien que des versions compressées comme TinyVLA [47] tentent d’accélérer ce processus, le temps de calcul reste un verrou majeur.

D’autre part, les approches modulaires découplent l’exécution en confiant l’interprétation sémantique de l’invite textuelle (par exemple, identifier l’aliment à acquérir) à un modèle de vision-langage ou à un grand modèle de langage (LLM) [48–51], laissant la localisation géométrique fine à des modules dédiés. Ce découplage préserve l’interprétabilité du système et permet de remplacer indépendamment chaque composant. Pour la segmentation fine des objets, le modèle fondateur *Segment Anything* (SAM) de Kirillov et al. [52] a démontré d’excellentes capacités de généralisation zéro-shot sans réentraînement. Plus récemment, sa troisième version (*Segment Anything 3* ou SAM3) [53] a introduit la segmentation par concept, permettant de localiser toutes les instances correspondant à une requête sémantique en une seule inférence. Enfin, le modèle SAM2 de Ravi et al. [54] permet d’étendre cette logique dans le domaine temporel en assurant la propagation cohérente et le suivi des masques sémantiques le long du flux d’images, préparant ainsi la reconstruction 3D de la scène.

2.2.2 Reconstruction RGB-D et représentation par nuages de points

Une fois le masque sémantique obtenu en deux dimensions, la rétroprojection géométrique de ses pixels dans l'espace cartésien, facilitée par les caméras RGB-D, permet de générer des nuages de points tridimensionnels de la cible et des obstacles. Se pose alors la question de la représentation géométrique optimale pour conditionner les politiques génératives. À cet égard, le conditionnement direct sur des nuages de points s'est imposé comme une alternative solide aux images brutes, car il capture explicitement les relations spatiales tridimensionnelles de la scène [55, 56]. Cette efficacité est illustrée par la politique *3D Diffusion Policy* [28] et le modèle fondationnel FP3 [57], qui démontrent une excellente généralisation géométrique avec un nombre réduit de démonstrations expertes.

Toutefois, l'acquisition d'un nuage de points à partir d'une unique caméra embarquée reste vulnérable aux occlusions physiques induites par les mouvements de l'effecteur ou du bras articulé lui-même. Pour y faire face, des méthodes comme CordViP de Fu et al. [58] proposent de reconstruire les zones masquées en combinant proprioception et estimation de pose 6D, restaurant ainsi la géométrie de la zone d'interaction. Bien que certains travaux suggèrent que des approches bidimensionnelles enrichies en pseudo-3D puissent parfois suffire [59, 60], l'utilisation de nuages de points 3D bruts présente l'avantage de ne requérir aucun modèle CAO préalable des objets de la scène, ce qui s'avère indispensable pour manipuler des formes complexes ou non structurées.

2.2.3 Cartes persistantes face aux occlusions et pertes de détection

Néanmoins, la perception instantanée en direct est insuffisante lors de tâches robotiques interactives dans des environnements dynamiques. Dès que le robot effectue une approche, son propre corps engendre des occlusions physiques massives, masquant temporairement la cible ou les obstacles proches [61]. Une politique fonctionnant purement en boucle ouverte et sans mémoire spatiale serait alors incapable de maintenir sa consigne de mouvement. Pour résoudre cette limite, la cartographie d'environnements dynamiques propose d'intégrer des graphes de facteurs (à l'instar de DynoSAM [62]) afin de séparer la géométrie statique des trajectoires d'objets mobiles. De même, des approches probabilistes comme SE(3)-PoseFlow de Jin et al. [63] utilisent le *Flow Matching* pour estimer les distributions de poses sous forte occlusion.

Afin de stabiliser durablement le signal de conditionnement fourni au planificateur nominal, l'intégration d'une couche de cartographie persistante (telle qu'une grille de voxels locale ou une carte d'occupation historique) s'avère indispensable. En accumulant et en filtrant les

nuages de points capturés au fil du temps, cette mémoire spatiale maintient une représentation géométrique tridimensionnelle fiable des obstacles et de la cible, même lors de pertes temporaires de détection visuelle. Ce mécanisme garantit la continuité géométrique du signal envoyé au planificateur, constituant une brique essentielle de l’architecture développée pour naviguer en toute sécurité dans les zones confinées.

2.3 Représentations géométriques pour la collision

Objectif de la section : Évaluer les méthodes de modélisation de l’encombrement spatial du robot et des obstacles pour formuler des métriques de distance différentiables.

2.3.1 Champs de distance de la scène et limites computationnelles

Imposer une contrainte d’évitement active suppose de disposer d’une métrique de distance différentiable entre le robot et les obstacles de son environnement. Une première famille de méthodes centre cette métrique sur la scène en discrétisant l’espace de travail. Les champs de distance signés euclidiens (*Euclidean Signed Distance Fields*, ESDF) encodent, en chaque point de l’espace, la distance à la surface de l’obstacle le plus proche. Formellement, le champ associe à un point $\mathbf{p} \in \mathbb{R}^3$ sa distance signée à la surface d’un obstacle $\mathcal{O} \subset \mathbb{R}^3$:

$$d(\mathbf{p}) = s_{\mathcal{O}}(\mathbf{p}) \min_{\mathbf{q} \in \partial\mathcal{O}} \|\mathbf{p} - \mathbf{q}\|_2, \quad s_{\mathcal{O}}(\mathbf{p}) = \begin{cases} -1, & \mathbf{p} \in \mathcal{O}, \\ +1, & \text{sinon,} \end{cases} \quad (2.6)$$

où la distance est ainsi comptée négativement à l’intérieur du volume de l’obstacle et positivement à l’extérieur, la frontière $\partial\mathcal{O}$ correspondant au niveau zéro $d(\mathbf{p}) = 0$. Un tel champ de distance théorique vérifie l’équation eikonale :

$$\|\nabla d(\mathbf{p})\|_2 = 1, \quad (2.7)$$

indiquant que le gradient du champ est unitaire et pointe dans la direction d’éloignement la plus rapide de l’obstacle, ce qui fournit directement la direction d’évitement.

Pour évaluer numériquement ce champ de distance de la scène en robotique, trois grands paradigmes coexistent :

1. **Le pré-calcul hors ligne (Offline) :** Pour des environnements statiques et connus, l’ESDF est échantillonné à l’avance sur une grille de voxels à partir du modèle CAO de la scène. Cette méthode, utilisée historiquement dans des planificateurs comme CHOMP [4], permet des requêtes de distance en temps constant $\mathcal{O}(1)$ lors de la plani-

fication, mais s'avère incapable de s'adapter au moindre changement d'environnement ou à l'apparition d'obstacles imprévus.

2. **La reconstruction en ligne par grilles volumétriques incrémentales (Online Grids) :** Les bibliothèques de cartographie en direct, à l'instar d'OctoMap [64], Voxelblox [65] ou nvblox [66], maintiennent et mettent à jour incrémentalement l'ESDF à partir des flux de données de capteurs RGB-D. Pour accroître la rapidité ou la précision de ces grilles, des architectures optimisées comme FIESTA [67] (optimisant la vitesse de calcul pour la planification de drones) ou Voxfield [68] (qui exploite des TSDF non projectifs pour améliorer la fidélité de la distance) ont été développées. Bien que ces grilles s'adaptent aux scènes dynamiques et exploitent le parallélisme des GPU [69], leur coût de mise à jour tridimensionnelle reste prohibitif et dépend fortement de la résolution choisie, limitant leur réactivité.
3. **Les champs de distance neuronaux continus en ligne (Online Continual Neural SDF) :** Afin de s'affranchir de la discrétisation par grilles volumétriques, des approches de reconstruction continue en ligne, comme iSDF d'Ortiz et al. [70], entraînent en temps réel un réseau de neurones de coordonnées sur le flux sensoriel actif pour apprendre une représentation continue et différentiable de la scène. Ce concept s'inscrit dans la lignée des frameworks de SLAM implicite en temps réel tels qu'iMAP [71] et sa variante hiérarchique NICE-SLAM [72]. Cependant, ces modèles neuronaux souffrent d'une latence de convergence lors de changements rapides de la scène et de risques d'oubli catastrophique, ce qui complique leur intégration directe dans une boucle de contrôle sécuritaire à haute fréquence.

Ces limitations computationnelles partagées motivent le développement de représentations alternatives centrées sur le robot plutôt que sur la scène.

2.3.2 Primitives conservatives et modèles géométriques simplifiés

Pour contourner la lourdeur du calcul des champs de distance globaux de la scène, une seconde famille de méthodes choisit plutôt de simplifier la géométrie du robot lui-même. L'approche classique consiste à encapsuler les différents liens de la chaîne cinématique dans des volumes englobants simples, tels que des sphères ou des capsules, dont les tests de collision analytiques se parallélisent très efficacement sur GPU. La bibliothèque cuRobo de Sundaralingam et al. [73] exploite cette idée pour générer des mouvements sûrs en quelques dizaines de millisecondes, surpassant de près de deux ordres de grandeur les solveurs classiques de l'état de l'art. Récemment, ces enveloppes ont été adaptées pour prendre en compte l'outil monté à l'effecteur en temps réel [74]. Néanmoins, le revers de cette approximation géométrique

conservative est qu'elle tend à surestimer grandement l'encombrement réel du robot. Cette perte d'espace libre exploitable restreint la capacité du robot à évoluer dans des zones confinées, ce qui est problématique pour des tâches d'assistance physique comme l'alimentation.

2.3.3 SDF du robot, RDF et approximation par polynômes de Bernstein

Afin de concilier la précision géométrique et la rapidité de calcul, un changement de paradigme récent propose de déplacer le champ de distance de la scène vers le robot en modélisant directement la géométrie du bras articulé. Les champs de distance du robot (*Robot Distance Fields*, RDF), introduits par Li et al. [75], approchent la SDF de chaque lien du robot par des polynômes de Bernstein. Pour une coordonnée spatiale normalisée $\mathbf{u} = (u_1, u_2, u_3) \in [0, 1]^3$, la distance signée associée à un lien l s'écrit comme un produit tensoriel 3D de la base polynomiale de Bernstein de degré n :

$$\bar{d}_l(\mathbf{u}) = \sum_{i=0}^n \sum_{j=0}^n \sum_{k=0}^n c_{ijk}^l B_{i,n}(u_1) B_{j,n}(u_2) B_{k,n}(u_3), \quad (2.8)$$

où les coefficients c_{ijk}^l , appris hors ligne à partir du modèle CAO, encodent précisément la forme géométrique tridimensionnelle du lien. Les polynômes de base de Bernstein de degré n sont définis analytiquement par :

$$B_{i,n}(u) = \binom{n}{i} u^i (1-u)^{n-i}. \quad (2.9)$$

Puisque la distance approchée $\bar{d}_l(\mathbf{u})$ est une fonction polynomiale, son gradient par rapport aux coordonnées spatiales s'obtient analytiquement sans nécessiter de coûteuses approximations par différences finies. En particulier, la dérivée d'un polynôme de base par rapport à la coordonnée normalisée u s'écrit simplement sous la forme :

$$\frac{d}{du} B_{i,n}(u) = n (B_{i-1,n-1}(u) - B_{i,n-1}(u)), \quad (2.10)$$

avec la convention que $B_{-1,n-1}(u) = 0$ et $B_{n,n-1}(u) = 0$. Cette propriété permet un calcul exact et instantané du gradient spatial sur GPU, pour une empreinte mémoire négligeable.

Contrairement aux représentations neuronales alternatives (telles que les *Neural SDF* [76]) qui souffrent de temps d'inférence plus élevés et d'un manque de régularité mathématique, l'approximation polynomiale de Bernstein fournit une structure différentiable et continue. Ses gradients pouvant être interrogés directement par rapport aux points de l'environnement, Govoni et al. [77] les ont exploités pour l'évitement réactif dans un cadre de commande

de tâches. Toutefois, l’usage d’opérateurs de composition non lisses (comme le minimum discret sur les distances des différents liens ou le *soft-min* pour regrouper les contributions de l’environnement) implique que la fonction de distance globale du robot n’est pas strictement de classe $\mathcal{C}^2$, une limitation théorique dont il faut tenir compte lorsque cette distance est exploitée comme fonction barrière dans une loi de commande.

2.4 Sécurité réactive et Control Barrier Functions

Objectif de la section : Analyser les lois de commande réactives issues de la théorie du contrôle afin d’imposer une contrainte de sécurité en vitesse sous certaines hypothèses.

2.4.1 Champs de potentiel et méthodes réactives classiques

La sécurité réactive consiste à modifier localement et en temps réel le mouvement du robot directement lors de son exécution sur la plateforme physique. Historiquement, la première formulation repose sur des forces virtuelles à travers la méthode des champs de potentiel artificiels (APF) introduite par Khatib [78]. Le principe est d’associer à la cible une force attractive et aux obstacles des forces répulsives. Bien que cette approche soit très légère en calcul et s’implémente facilement en ligne, elle souffre de limites théoriques majeures : l’absence de garanties de stabilité pour des dynamiques complexes, des oscillations à proximité des obstacles et un blocage fréquent dans des minima locaux induits par des géométries concaves. Néanmoins, le concept reste exploité dans des architectures génératives récentes pour guider des politiques d’imitation vers des trajectoires sûres [79].

2.4.2 Fonctions barrières de contrôle et invariance avant

Pour surmonter le manque de rigueur mathématique des champs de potentiel, les fonctions barrières de contrôle (*Control Barrier Functions*, CBF), formalisées par Ames et al. [80] sur la base des certificats de barrière de Wieland et Allgöwer [81], offrent un cadre d’invariance rigoureux. Considérons un système dynamique non linéaire affine en contrôle de la forme :

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}, \quad (2.11)$$

où $\mathbf{x} \in \mathbb{R}^d$ désigne l’état et $\mathbf{u} \in \mathbb{R}^m$ l’action de commande. On définit un ensemble de sécurité $\mathcal{C}$ comme le sous-ensemble de l’espace d’état où une fonction scalaire continûment différentiable $h : \mathbb{R}^d \rightarrow \mathbb{R}$ reste positive :

$$\mathcal{C} = \{\mathbf{x} \in \mathbb{R}^d : h(\mathbf{x}) \geq 0\}, \quad (2.12)$$

la frontière $h(\mathbf{x}) = 0$ marquant le contact ou la limite de sécurité avec l'obstacle. L'ensemble $\mathcal{C}$ est dit invariant vers l'avant si toute trajectoire débutant dans $\mathcal{C}$ y demeure indéfiniment. En exploitant les dérivées de Lie $L_f h(\mathbf{x}) = \nabla h(\mathbf{x})^\top f(\mathbf{x})$ et $L_g h(\mathbf{x}) = \nabla h(\mathbf{x})^\top g(\mathbf{x})$, la condition d'invariance se traduit par l'existence d'une fonction de classe $\mathcal{K}$ (typiquement linéaire avec un gain $\kappa > 0$) telle que :

$$L_f h(\mathbf{x}) + L_g h(\mathbf{x}) \mathbf{u} \geq -\kappa h(\mathbf{x}). \quad (2.13)$$

Lorsque le contrôle du robot s'effectue directement en vitesse articulaire (degré relatif un, où l'état correspond à la configuration articulaire $\mathbf{x} = \mathbf{q}$ et l'action à la vitesse $\mathbf{u} = \dot{\mathbf{q}}$), la dynamique se réduit à $\dot{\mathbf{q}} = \mathbf{u}$. La condition barrière (2.13) se simplifie alors sous la forme linéaire suivante :

$$\nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq -\kappa h(\mathbf{q}). \quad (2.14)$$

Dans le cadre de l'évitement de collision articulé, la fonction barrière $h_i(\mathbf{q})$ associée à un point critique i du robot s'exprime directement à partir de la distance physique aux obstacles $d_{\text{robot},i}(\mathbf{q})$, évaluée via le modèle de Bernstein (SDF locale), minorée d'une marge de sécurité minimale d_{safe} :

$$h_i(\mathbf{q}) = d_{\text{robot},i}(\mathbf{q}) - d_{\text{safe}}. \quad (2.15)$$

La condition (2.14) étant affine par rapport aux vitesses articulaires $\dot{\mathbf{q}}$, elle peut s'intégrer sous forme de contraintes linéaires au sein d'un programme quadratique (QP-CBF) résolu à chaque cycle de commande [82] :

$$\dot{\mathbf{q}}^* = \arg \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{nom}}\|_2^2 \quad \text{s.c.} \quad \nabla h_i(\mathbf{q})^\top \dot{\mathbf{q}} \geq -\kappa h_i(\mathbf{q}), \quad \forall i \in \{1, \dots, N_c\}, \quad (2.16)$$

où $\dot{\mathbf{q}}_{\text{nom}}$ représente la consigne nominale et N_c désigne le nombre de contraintes actives (les points critiques les plus proches de l'environnement). Le programme quadratique (2.16) agit ainsi comme un filtre de sécurité réactif en projetant la vitesse nominale sur le demi-espace admissible le plus proche. Des extensions étendent cette formulation aux systèmes mécaniques de degré relatif deux (commande en couples ou accélérations) [83].

2.4.3 Limites des CBF sous perception bruitée et contrôle discret

Néanmoins, les garanties d'invariance théorique établies en temps continu et sous l'hypothèse d'un modèle géométrique parfait se dégradent fortement lors de leur déploiement sur une plateforme physique réelle. En effet, trois facteurs limitent l'invariance absolue :

1. **La discrétisation temporelle** : La boucle de commande s'exécute à fréquence finie, ce qui peut provoquer des franchissements de la barrière entre deux pas de calcul. Cette

limite a conduit au développement des formulations en temps discret (DT-CBF) de la condition barrière [84].

2. **Les saturations matérielles :** Les limites de vitesse et de couple des actionneurs restreignent le domaine d’admissibilité du QP. Si la contrainte CBF exige une accélération ou une vitesse supérieure aux capacités physiques du robot, le programme quadratique devient infaisable [85], compromettant la sécurité.
3. **Le bruit de perception et l’erreur d’interface :** Le calcul du gradient sur des nuages de points réels et bruités introduit des fluctuations dans les contraintes. De plus, le suivi de la consigne de vitesse $\dot{\mathbf{q}}^*$ par un contrôleur en position bas niveau engendre des retards et des dérives cinématiques.

Par conséquent, dans des applications réelles non structurées, la loi de commande CBF doit être traitée comme un filtre de sécurité pragmatique et probabiliste, plutôt que comme une garantie formelle infaillible, ce qui justifie l’utilisation de marges de sécurité conservatives.

2.5 Couplage entre modèles génératifs et filtres de sécurité

Objectif de la section : Examiner les différentes stratégies d’unification de l’IA générative et des lois de commande sécuritaires afin de positionner la méthode de découplage retenue.

Pour allier l’expressivité et la multimodalité des modèles génératifs avec la rigueur des garanties de non-collision, deux grandes stratégies d’unification se distinguent dans la littérature : l’intégration des contraintes directement au cœur du processus de génération, ou l’application d’un filtre de sécurité en aval lors de l’exécution de la trajectoire.

2.5.1 Contraintes intégrées à la génération

La première stratégie injecte la contrainte de sécurité au sein même du processus d’intégration de l’équation différentielle (guidage intra-ODE). Par exemple, SafeFlow de Dai et al. [86] formule des fonctions barrières adaptées au *Flow Matching* afin de confiner le flux de trajectoires dans l’espace libre sans nécessiter de réentraînement du réseau. Dans la même ligne, SafeFlowMatcher [87] propose un intégrateur de type prédiction-correction certifié pour converger en temps fini vers la zone sûre. De son côté, CASF (Constrained Action Streaming Flows) de Long et al. [88] déforme localement la métrique riemannienne de l’espace de commande pour rejeter les trajectoires s’approchant des frontières d’obstacles. D’autres variantes récentes adaptent ces concepts sous forme de contraintes de Lyapunov ou de barrières stochastiques intégrées aux processus de diffusion [89, 90], ou s’appuient sur des optimisations unifiées pour garantir l’admissibilité dynamique des mouvements [91–93].

Cependant, bien qu’élégantes en simulation, ces méthodes intra-génération se heurtent à une limite structurelle majeure : la dérive distributionnelle (*distributional drift*). En forçant le processus d’intégration à traverser des états latents qui s’écartent de la distribution des données expertes vue à l’entraînement, ces contraintes perturbent la dynamique interne du modèle. Par conséquent, elles dégradent fréquemment la qualité globale du mouvement généré et diminuent le taux de succès sur la tâche. De surcroît, le coût computationnel d’une résolution de contraintes répétée à chaque étape d’intégration exclut généralement leur utilisation pour du contrôle réactif en boucle fermée.

2.5.2 Filtres de sécurité appliqués à l’exécution

Pour préserver l’intégrité du modèle génératif, la seconde stratégie maintient la phase de génération nominale totalement inchangée et n’applique les contraintes de sécurité qu’en aval, lors de la commande physique du robot. L’archétype de ce paradigme est le cadre PACS (*Path-Consistent Safety filtering*) proposé par Römer et al. [94], qui utilise des méthodes d’atteignabilité ensembliste pour projeter la commande nominale sur un espace admissible en environnement dynamique. Ce type de filtrage externe préserve la distribution nominale apprise tout en garantissant la sécurité physique, ce qui permet d’atteindre des taux de réussite supérieurs de près de 68 % par rapport à une intégration directe des contraintes.

Dans cette perspective, d’autres approches proposent d’appliquer des corrections locales réactives (comme FlowCorrect [95]) pour adapter à la volée les trajectoires nominales à partir de signaux sporadiques, résolvant des cas d’échecs complexes sans réentraîner le réseau. Enfin, des filtres temporels spécifiques permettent également de compenser les délais d’inférence inhérents aux grands modèles génératifs [96, 97].

2.5.3 Découplage entre intention nominale et sécurité réactive

L’analyse croisée de ces deux familles d’approches justifie l’architecture découplée mise en œuvre dans ce travail. Plutôt que d’entrelacer la sécurité et la génération nominale, nous optons pour une isolation fonctionnelle complète entre :

1. **Un planificateur global en boucle ouverte** basé sur le *Flow Matching*, chargé de générer l’intention nominale à long terme sans subir de dérive distributionnelle.
2. **Un filtre de sécurité réactif local** basé sur une projection QP-CBF, s’exécutant à haute fréquence en ligne pour intercepter les collisions physiques sur le robot.

Ce découplage fonctionnel concilie efficacement la richesse et la multimodalité des trajectoires apprises par le modèle génératif avec la réactivité et la rigueur théorique indispensables pour

garantir la sécurité physique sur une plateforme réelle. Ce principe de séparation constitue le fil conducteur de la méthodologie proposée dans ce mémoire.

2.6 Synthèse critique et positionnement

Objectif de la section : Résumer les lacunes de l'état de l'art, formaliser l'hypothèse de recherche et présenter un tableau comparatif synthétique des architectures de la littérature.

2.6.1 Lacunes identifiées dans la littérature

L'analyse croisée de la littérature scientifique contemporaine met en évidence trois lacunes majeures :

1. **L'hypothèse d'une perception parfaite :** Les frameworks actuels validant l'alliance du *Flow Matching* et des CBF supposent des obstacles analytiques parfaits connus a priori, se montrant inadaptés face aux nuages de points 3D bruts et bruités.
2. **L'absence de persistance temporelle des modèles génératifs :** Les planificateurs génératifs souffrent d'une vulnérabilité cinématique lors d'occlusions temporaires de la cible, faute d'une couche cartographique persistante capable de stabiliser les points de conditionnement.
3. **Le coût numérique des représentations globales :** Les approches basées sur la mise à jour de champs de distance globaux ou de *Neural SDF* s'avèrent trop lourdes pour garantir une boucle de rétroaction à haute fréquence face à des obstacles imprévus.

2.6.2 Positionnement de l'architecture proposée

Pour répondre à ces limites, ce mémoire soutient la pertinence d'une *architecture découplée entre génération nominale et filtrage réactif de sécurité*. La trajectoire nominale est générée en boucle ouverte par un modèle *Flow Matching* conditionné par une observation géométrique persistante issue de la perception RGB-D et du suivi SAM2, tandis que la sécurité immédiate est déléguée à un filtre de projection CBF articulaire bas niveau s'appuyant sur une approximation polynomiale de Bernstein du SDF local du robot.

2.6.3 Hypothèse de recherche

L'hypothèse centrale de cette recherche s'énonce ainsi :

« L'utilisation d'un filtre SDF-CBF fondé sur un SDF local du robot, approximé par des polynômes de Bernstein et alimenté par une carte perceptuelle persistante, permet de corriger en temps réel les vitesses nominales générées par un modèle Flow Matching, afin d'améliorer l'exécution sécuritaire de trajectoires dans des environnements quasi-statiques, non structurés et inconnus, sans réentraîner le modèle génératif ni construire un champ de distance global de l'environnement.

»

Le Tableau 2.1 résume le positionnement de l'approche proposée vis-à-vis des frameworks de référence de la littérature.

Tableau 2.1 Tableau comparatif des frameworks de planification et de sécurité de l'état de l'art.

Cadre / Méthode	Modèle nominal	Sécurité	Représentation	Correction	Occlusion	Réactivité
Diffusion Policy	Diffusion	Implicite	Espace latent	Aucune / boucle ouverte	Non	Limitée par le débruitage
OmniGuide	Flow Matching	Souple	Grille / énergie	Guidage intra-ODE	Non	Moyenne
CASF	Streaming Flow	Souple	Neural SDF	Déformation intra-ODE	Partielle	Élevée mais coûteuse
SafeFlow- Matcher	Flow Matching	Explicite CBF	Obstacles analytiques	Correction intra-génération	Non	Élevée en simulation
Notre approche	Flow Matching	Explicite : projection SDF-CBF	Nuage filtré persistant + SDF robot	Correction vitesse articulaire à l'exécution	Oui, via persis- tance	Élevée, mesurée expérimen- talement

En somme, l'examen critique de la littérature scientifique met en évidence un compromis persistant entre l'expressivité des politiques génératives et la rigueur des algorithmes de préservation de la sécurité. Si le *Flow Matching* lissant et déterministe permet de capturer efficacement la multimodalité des mouvements experts tout en réduisant la latence propre aux processus de diffusion stochastique, son exécution en boucle ouverte au sein d'environnements réels et changeants demeure sujette aux incertitudes perceptives. Les méthodes contemporaines tentant d'incorporer des contraintes au cours de la phase de génération tendent à alourdir le temps de calcul ou à provoquer une dérive distributionnelle nuisible pour le succès de la tâche. Afin d'apporter une solution pragmatique à ces verrous technologiques, le Chapitre 4 détaillera la méthodologie de l'architecture découplée proposée dans ce travail. Nous y exposerons comment un pipeline de perception persistant s'articule de manière synchrone avec un filtre réactif basé sur une approximation par polynômes de Bernstein pour améliorer,

en temps réel et sous des contraintes vérifiables localement, l'exécution sécurisée définie au cœur de notre hypothèse de recherche.

CHAPITRE 3 MOTIVATION DE RECHERCHE ET OBJECTIFS

3.1 Contexte et problématique

3.2 Objectifs de recherche

L'intégration d'un filtre de sécurité SDF-CBF fondé sur un SDF local du robot permet de corriger en temps réel les vitesses nominales générées par un modèle Flow Matching, afin d'exécuter des trajectoires dans des environnements non vus tout en réduisant les risques de collision et en préservant l'intention de la trajectoire nominale.

3.3 Contexte et problématique

3.4 Portée et exclusions de la recherche

CHAPITRE 4 DESIGN DU SYSTÈME

4.1 Plateforme Expérimentale Et Architecture Globale

4.1.1 Scène robotique, capteur RGB-D, robot manipulateur

4.1.2 Architecture modulaire du système

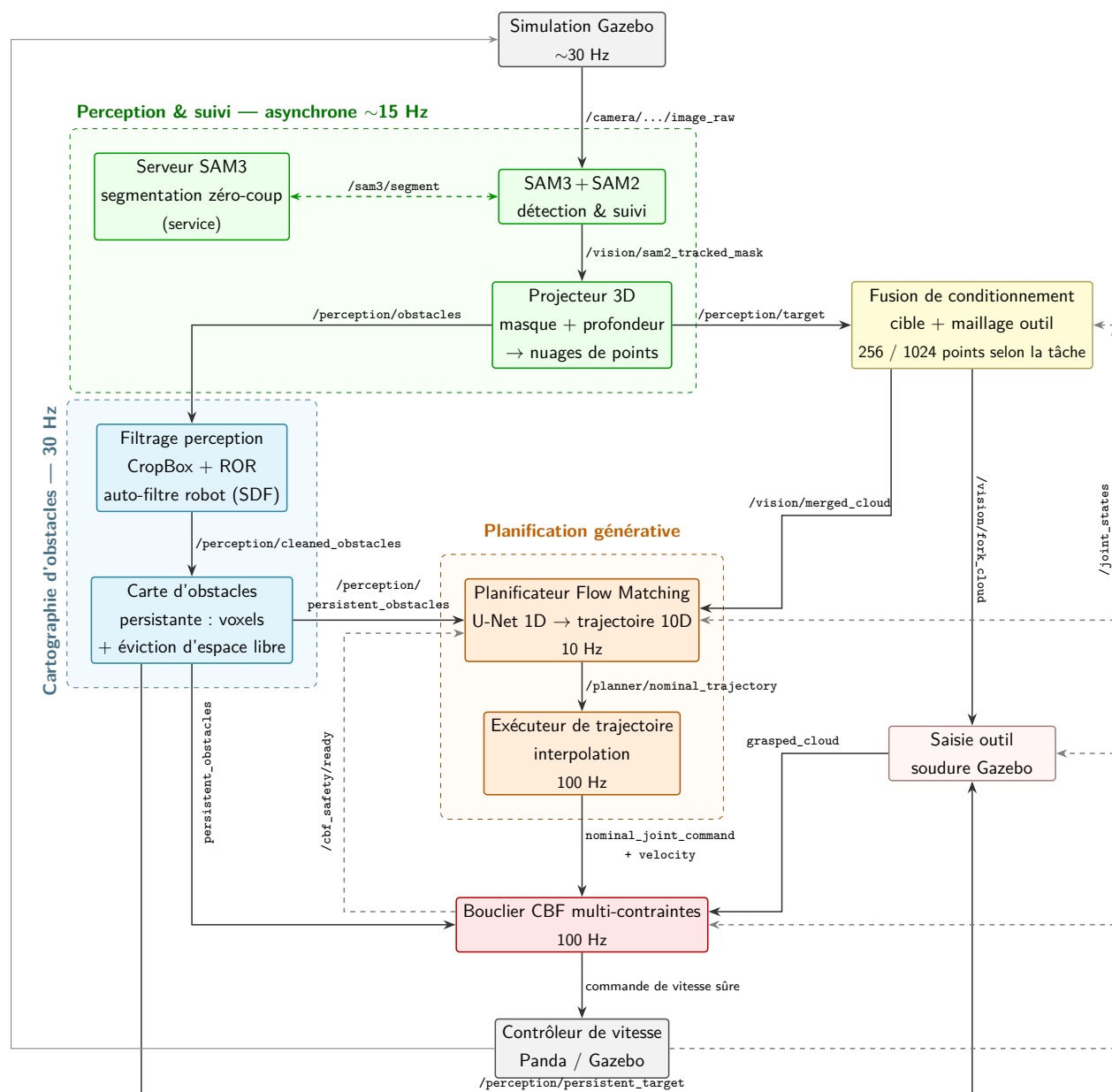


Figure 4.1 Architecture globale du système asynchrone.

4.2 Perception Sémantique et Reconstruction 3D

4.2.1 Extraction Sémantique

Afin de détecter les objets à détecter pour contraindre le modèle de génération de trajectoire utilisée, le modèle de fondation *Segment Anything Model 3* (SAM 3) de meta [53] est utilisé. Ce modèle permet de calculer la boîte englobante, la segmentation au niveau du pixel, nommé masque, ainsi qu'un score de confiance de détection à partir d'une photo RGB et d'un prompt. Le fait que le masque soit obtenu directement à la sortie du modèle permet par la suite à un modèle contenant moins de paramètres de faire le suivi des objets détectés par SAM 3, permettant ainsi de grandement diminuer le temps d'inférence pour le suivi temporel. Ce modèle de fondation a été entraîné sur plus de quatre millions de concepts textuels et 38 millions de phrases synthétiques, ce qui lui confère la capacité de détecter une quantité quasi illimitée d'objets. La figure 4.2 présente un exemple d'un masque obtenu suite à la demande *Pink Block*.



Figure 4.2 Exemple de masque obtenu par le modèle de segmentation SAM3 pour le prompt *Pink Block*

4.2.2 Suivi temporel de la cible

À partir du masque détecté par SAM 3 à la section 4.2.1, le suivi...

4.2.3 Projection RGB-D vers l'espace 3D

4.2.4 Séparation cible, obstacles et outil visible

4.2.5 Nettoyage spatial du nuage d'obstacles

4.3 Cartes Persistantes de la scène

4.3.1 Motivation : robustesse aux pertes de détection et occlusion

On veut pouvoir éviter les occlusions et les pertes de détection de SAM 3, en gardant en mémoire les surfaces des objets

4.3.2 Génération des cartes persistantes

4.3.3 Mise à jour des cartes persistantes

Conservation des points hors frame

Voxelisation de la map pour alléger les calculs

Suppression des doublons dans chaque Voxelisation

fusion avec la carte existante

jusqu'à la limite $\max_voxels$ (suppression des points par vieillissement temporel par la suite pour voir les points les plus proches et les nouveaux points)

Suppression des voxels supprimées si $z_{measure} - z_{voxel} > seuil$ (car la caméra voit là où le voxel était supposé caché)

4.4 Acquisition des données et représentation des démonstrations

Cette sous-section a comme objectif de présenter le protocole utilisé afin de collecter les données nécessaires pour entraîner le modèle d'IA générative ainsi que l'étape de prétraitement de ces données.

4.4.1 Protocole de collecte des démonstrations

Les données d'entraînement utilisées pour entraîner le modèle de Flow Matching ont été acquises de deux manières. La première d'entre elle est sur le robot et les capteurs réels et la seconde est issue de simulation avec la librairie Maniskill 3. Cette librairie a été choisie pour sa facilité d'utilisation et son moteur de calcul à l'état de l'art basé sur SAPIEN 3 [98]. Les

deux méthodes utilisent des environnements quasi identiques afin de récolter les données de la même manière. Pour garantir un protocole reproductible, l'échantillonnage des configurations est défini dans deux repères imbriqués plutôt que de façon qualitative : le repère robot $\mathcal{F}_b$ (base du manipulateur, fixe) et un repère assiette $\mathcal{F}_p$ centré sur l'assiette. L'assiette est placée successivement aux 9 positions d'une grille 3×3 exprimées directement dans $\mathcal{F}_b$ (pas de XXX m). Pour chacune, la cible occupe 5 positions disposées en quinconce dans $\mathcal{F}_p$, chacune assortie d'une orientation aléatoire. Cette grille produit $9 \times 5 = 45$ configurations, chacune réalisée une fois. La Figure 4.3 illustre cet échantillonnage. À chaque configuration, l'outil descend verticalement vers la cible (axe $-z$ de $\mathcal{F}_b$) en maintenant les dents verticales et alignées sur l'axe principal de la cible. La simulation (*ManiSkill 3*) réutilise ce même schéma mais l'élargit par une randomisation de hauteur couvrant un surensemble continu des configurations réelles.

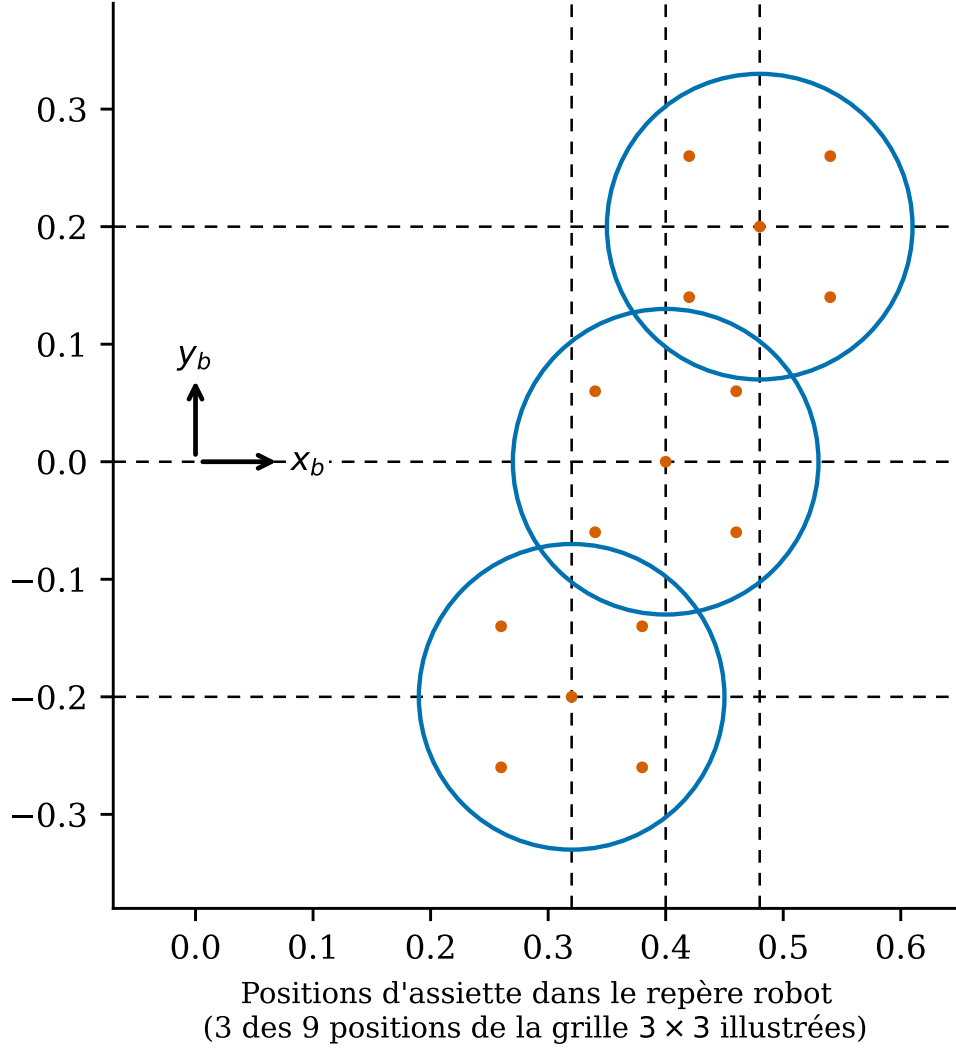


Figure 4.3 Protocole d'échantillonnage des cibles, vu de dessus dans le repère de base du robot $\mathcal{F}_b$. Les assiettes (26 cm de diamètre) se chevauchant, seules trois des neuf positions de la grille 3×3 sont illustrées (supérieur droit, centre, inférieur gauche), le centre de chacune étant marqué par une croix noire. Autour de chaque centre, les 5 points indiquent les positions de cible en quinconce.

Pour récolter les données réelles, le protocole détaillé suivant a été utilisé :

1. Installation de la caméra sur le support officiel de Franka au niveau de l'effecteur. Le support est en annexe
2. Positionnement des joints du robot à $\mathbf{q} = [0, -0.1259, 0, -2.1933, 0, 2.0648, 0.7855]$
3. Mettre le robot en mode Réflexe pour qu'il puisse être guider par un humain.
4. Réaliser la trajectoire robotique souhaitée pour aller chercher la cible et réaliser la

trajectoire à apprendre par le modèle.

5. Enregistrer les données dans un fichier JSON.

L'enregistrement des données se fait en utilisant 2 noeuds ROS distincts. Le premier noeud, écrit en C++ pour la rapidité d'exécution et de lecture, lis les données suivantes provenant du robot :

1. Pose du TCP dans le repère monde [1,7]. Les trois premières dimensions sont reliées à la position cartésienne et les 4 dernières à l'orientation sous la forme d'un quaternion.
2. Vitesse du TCP dans le repère monde [1,6]. Les trois premières position sont les vitesses cartésiennes et les 3 suivantes les vitesses angulaires.

L'ensemble de ces données sont publiées sous la forme de message ROS. Il sont par la suite suivies par un second noeud écrit en Python, qui suit aussi les données RGB-D de la caméra à l'effecteur. Ce noeud a la responsabilité d'enregistrer l'ensemble des données d'entraînement dans un fichier JSON. En plus des données du TCP, les données de la caméra ainsi que les données structurelles suivantes sont ajoutées afin d'organiser la base de données :

1. `step_number` : le numéro du jeu de données pour la trajectoire
2. `time_step` : le moment de la prise de données depuis le début de la trajectoire
3. `rgb` : le nom du fichier de la photo. Ce dernier est, par nomenclature, nommé `ee_rgb_step_{step_number}.png`.
4. `depth_npy` : le nom du fichier de la carte de profondeur. Ce dernier est, par nomenclature, nommé `ee_depth_step_{step_number}.npy`

La figure 4.4 représente l'architecture du système d'acquisition de données pour entraîner le modèle d'IA générative.

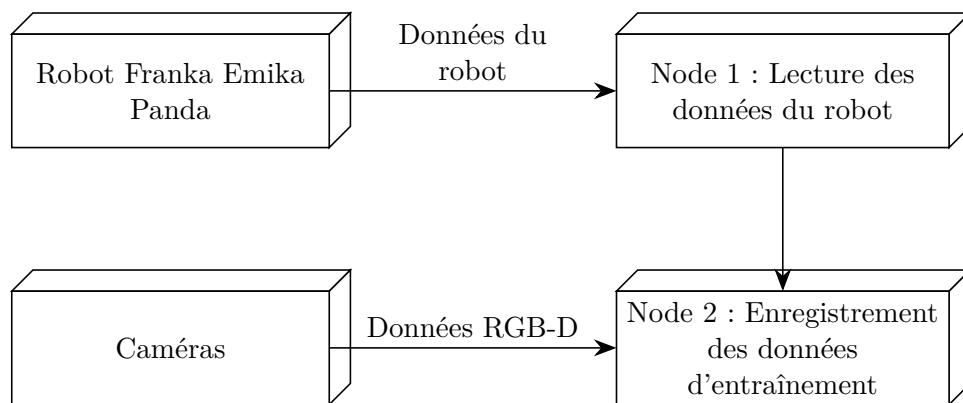


Figure 4.4 Architecture du système d'acquisition de données

Ces données sont ensuite enregistrées dans un dossier racine sous la forme d'un dossier nommé Trajectoire_X où X représente le numéro de la trajectoire enregistrée. Dans ce dossier chacun de ces dossiers se trouve un fichier JSON avec l'ensemble des données numériques et les noms des fichiers images ainsi qu'un dossier contenant l'ensemble des images et des fichiers de profondeurs. la représentation de l'architecture de l'enregistrement des fichiers est représentée sur la figure 4.5 :

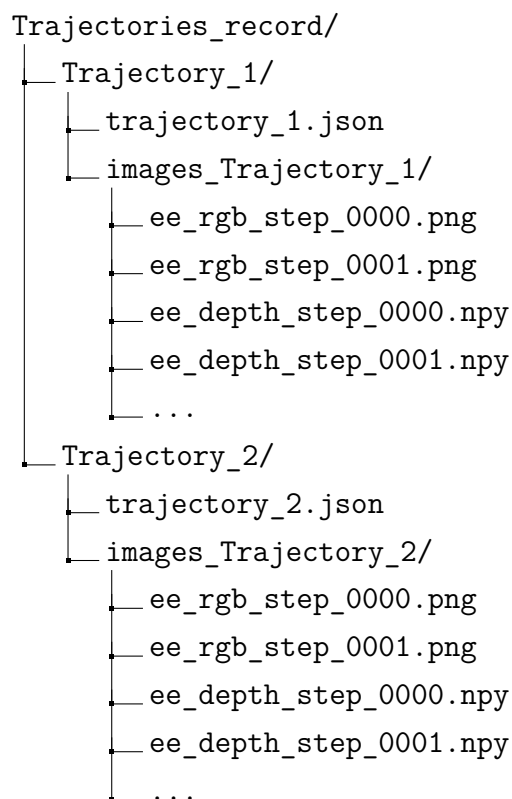


Figure 4.5 Directory structure of the PickPlace dataset execution tests.

4.4.2 Prétraitement des données

À partir des données enregistré en 4.4.1, plusieurs étapes de prétraitement sont réalisées hors ligne afin d'accélérer l'entraînement du modèle. Ces étapes sont listées et détaillées dans cette sous-section.

Détection avec un modèle de segmentation

La première étape consiste à détecter la cible dans la première image de chacune des trajectoires. Cette détection est réalisée avec le modèle à l'état de l'art Segment Anything 3

(SAM3) qui peut réaliser de la segmentation par prompt. Ainsi, une liste de cibles possibles est donnée en entrée au modèle et le masque de la cible dans l'image est obtenue en sortie. La liste en question est [*Pink block, Brocoli, Carot Slice, Chicken breast, Sushi*] Comme SAM3 prends que des images .jpg en entrée, il faut alors transformer la première image dans ce format. La raison de pourquoi on ne détecte pas la cible à chaque image est en raison de la limite physique du capteur de profondeur de la caméra utilisée qui est de 45 cm selon le détaillant.

Génération du nuage de point conditionnel

Considérant que le modèle de Flow Matching décrit dans la section 4.5 est conditionné sur les nuages de points de l'outil et de la cible, il est impératif de généré les nuages de points de chacun des étapes hors ligne afin d'accélérer le temps d'entraînement et de valider les données utilisées. Pour se faire, on utilise le masque généré à l'étape précédente, la carte de profondeur initiale, les angles dans les joints à chaque étape et le modèle CAO de l'outil.

La première étape consiste à générer un nuage de points de la cible dans le repère de la caméra. Pour y parvenir, une superposition du masque de la cible et de la carte de profondeur initiale est réalisée afin de conserver seulement les points de la cible. Par la suite, on calcul la position cartésiennes de chacun des points présents dans le masque en utilisant les équations ci-dessous :

$$X_c = \frac{Z}{f_x}(u - c_x) \quad (4.1)$$

$$Y_c = \frac{Z}{f_y}(v - c_y) \quad (4.2)$$

$$Z_c = Z \quad (4.3)$$

où f_x et f_y sont les longueurs focales de la caméra et c_x et c_y sont les coordonnées du point principal de la caméra. Les paramètres intrinsèques de la caméra D415 utilisée ont été obtenues au prêt du fabricant. Le nuage de points résultant est ensuite positionné dans le repère de la base du robot en utilisant la relation (4.4) où $\mathbf{T}_{cible}^b$ est la matrice de transformation entre la cible et la base du robot. Cette relation peut être déduite de la figure 4.6 où est représentée l'ensemble des repères nécessaires au système : $\{b\}$ le repère de la base du robot, $\{c\}$ pour le repère de la caméra au niveau de l'effecteur, $\{e\}$ pour le repère du TCP et $\{o\}$ pour le repère de l'outil utilisé. Ce dernier est finalement sous-échantillonner à 512 points en utilisant la stratégie d'échantillonnage par le point le plus éloigné.

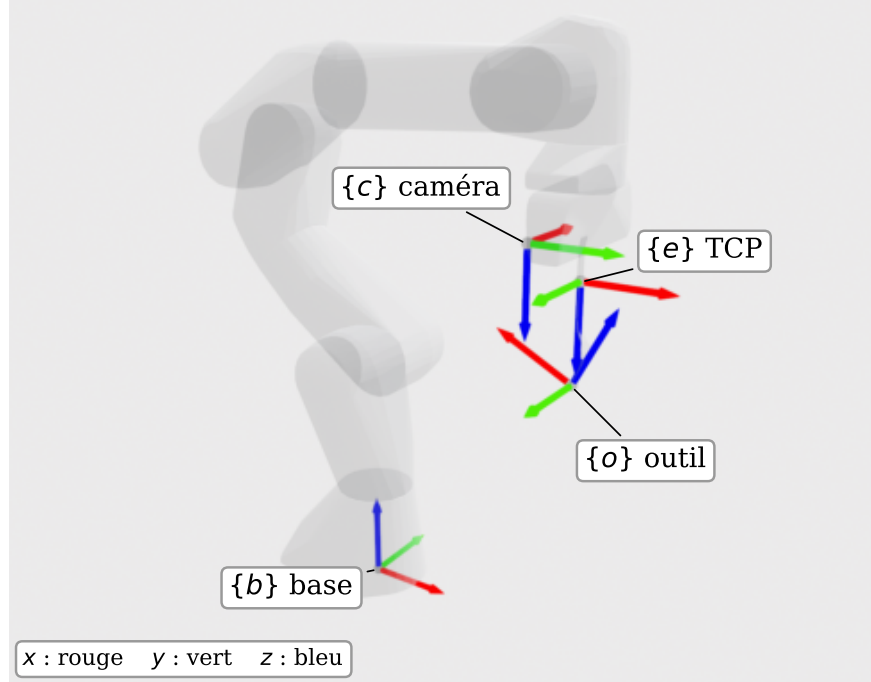


Figure 4.6 Représentation schématique des frames des différents repères utilisés

$$\mathbf{T}_{cible}^b = \mathbf{T}_e^b \mathbf{T}_c^e \mathbf{T}_{cible}^c \quad (4.4)$$

La seconde étape consiste à générer le nuage de points de l'outil. En supposant que le modèle CAD de l'outil est disponible, il est possible d'obtenir son nuage de points en utilisant la librairie `trimesh` avec la fonction `trimesh.sample.sample_surface`. Pour positionner ce nuage de points, et connaissant la pose du repère de l'outil dans le repère du TCP du robot, l'équation (4.5) est utilisée.

$$\mathbf{T}_f^b = \mathbf{T}_e^b \mathbf{T}_o^e \quad (4.5)$$

à noter que la transformation $\mathbf{T}_o^e = \mathbf{I}$ dans le cas où l'outil est le TCP. Comme pour la cible, le nuage de point ainsi positionné est par la suite sous-échantionné à 512 points en utilisant la même stratégie. Les deux nuages de points résultants sont finalement fusionnés ensemble et translatés au niveau du repère de l'outil $\{o\}$. Le nuage conditionnel compte ainsi 1024 points pour la tâche d'alimentation (512 points pour la cible et 512 pour l'outil). Pour la tâche de saisie et dépôt (*pick-and-place*), le nuage fusionné compte également 1024 points, répartis entre la pince (40%), le cube à saisir (20%) et la boîte de dépôt (40%), le sous-échantillonnage final utilisant la même stratégie du point le plus éloigné. Ce nuage de points final est ensuite enregistré sous le nom de `Merged_{step_number}` dans le dossier

Merged_Trajectory_X. Finalement, les fichiers JSON provenant de l'étape précédente sont copiés dans leur dossier du jeu de données prétraités respectif et la pose de l'outil dans le repère monde $\{b\}$ y est ajoutée. L'architecture depuis le dossier racine du jeu de données ainsi créée est représentée à la figure 4.7.

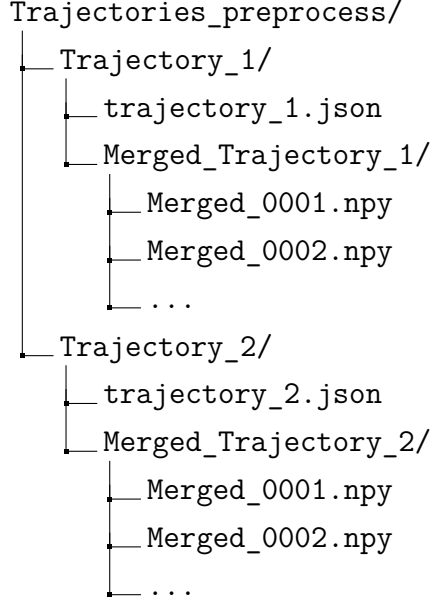


Figure 4.7 Structure du dossier du jeu de données des trajectoires prétraités

4.5 Génération de trajectoire par Flow Matching

Cette section traite de l'ensemble de l'architecture et des hyperparamètres utilisés pour entraîner le modèle de Flow Matching conditionnel afin de générer les trajectoires robotiques.

4.5.1 Formulation mathématique du Flow Matching conditionnel

Présenter la provenance et les mathématiques

$$\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x)) \quad (4.6)$$

$$X_t = (1 - t)X_0 + tX_1 \quad (4.7)$$

$$\mathcal{L}(\theta) = \mathbb{E}_{t, X_0, X_1} \|u_t^\theta(X_t) - (X_1 - X_0)\|^2, \text{ où } t \sim U[0, 1], X_0 \sim \mathcal{N}(0, 1), X_1 \sim q \quad (4.8)$$

4.5.2 Architecture du modèle

Présenter les entrants, les extrants, l'architecture du modèle (EMA, normalisation, MLP pour encodeur, application du conditionnement des trajectoires), comment il résout l'ODE (avec approximation d'Euler) et comment j'obtiens les positions finales

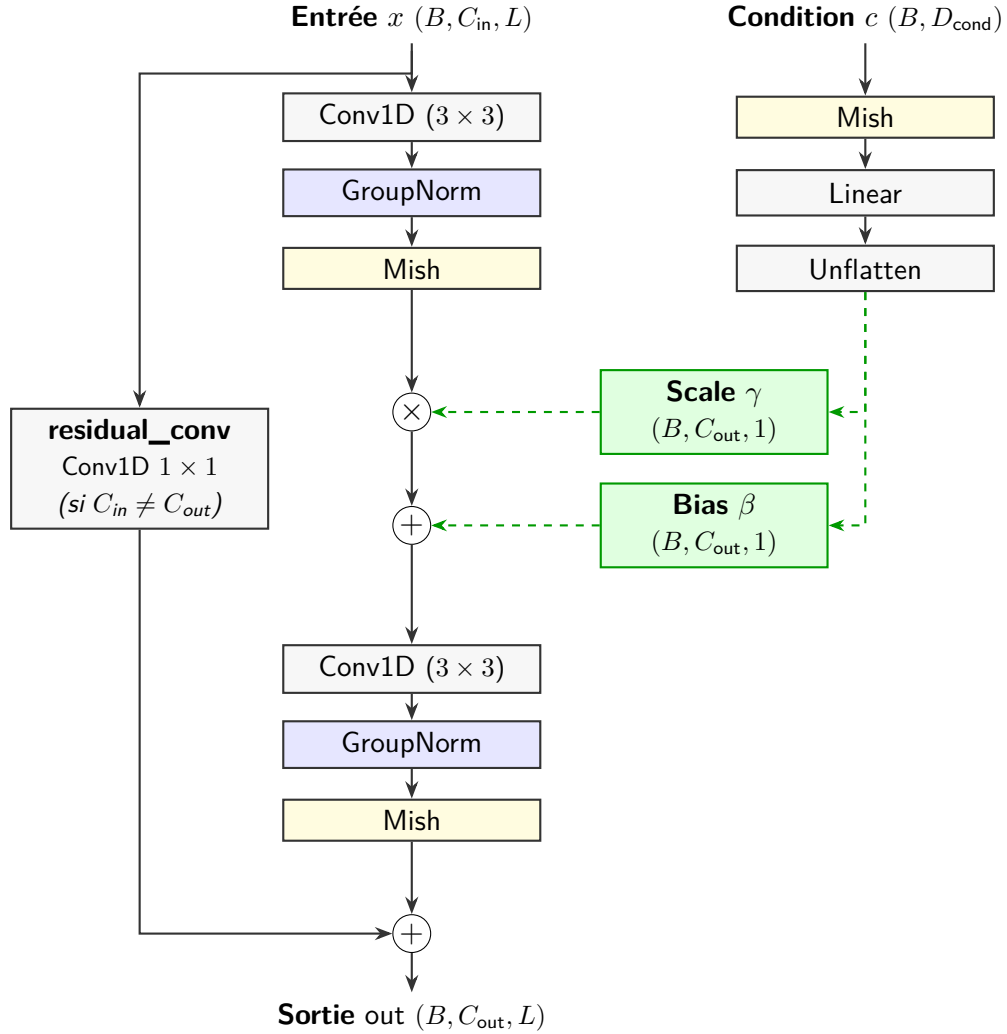


Figure 4.8 Architecture du module **ConditionalResidualBlock1D** avec un tracé orthogonal à angle droit unique vers les entrées latérales.

$$\text{Mish}(x) = x \cdot \tanh(\ln(1 + e^x)) \quad (4.9)$$

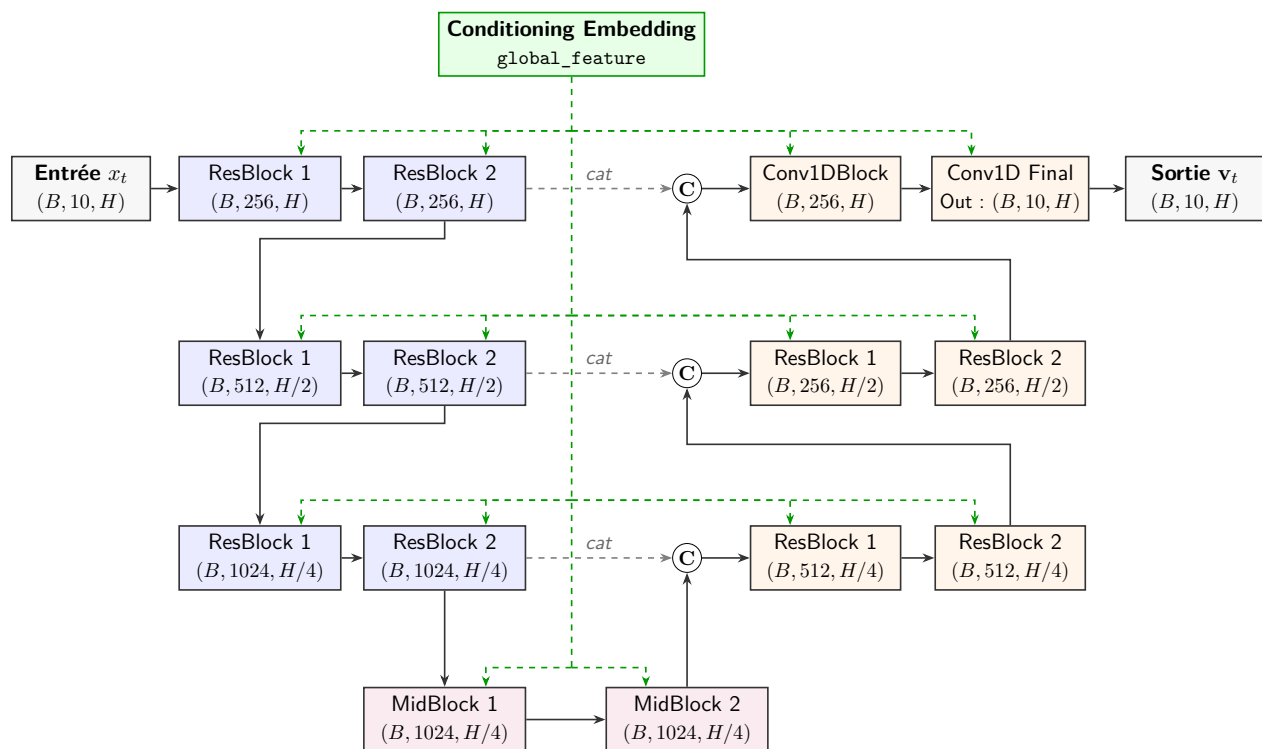


Figure 4.9 Architecture globale finale du ConditionalUnet1D au style épuré, corrigée de tout dépassement de tracé.

4.5.3 Entraînement du modèle

Présentation des hyperparamètres choisis et de la Loss curve (finalement loss curve pas si importante comme on présente une méthodologie).

4.6 Conversion et exécution de la trajectoire nominale

4.6.1 Conversion outil - TCP

Mettre une image des repères TCP - Outil et de la matrice de Transformation et comment on fait pour calculer la pose du TCP à contrôler en fonction de la pose de l'outil

4.6.2 Cinématique inverse pour obtenir la trajectoire articulaire nominale

La trajectoire générée par *Flow Matching* décrit le mouvement du lien contrôlé dans l'espace cartésien : sur l'horizon H , le modèle produit une séquence de poses que la conversion outil-TCP de la sous-section précédente ramène en poses désirées du repère TCP, ${}^b\mathbf{T}_k^* \in SE(3)$, $k = 1, \dots, H$. Le robot étant commandé dans l'espace articulaire, chaque pose doit être convertie en une configuration $\mathbf{q}_k \in \mathbb{R}^7$ par cinématique inverse. Seules les sept articulations du bras interviennent dans la tâche : les deux articulations de la pince complètent le vecteur de configuration du modèle mais n'influent pas sur la pose du TCP, et la préhension est commandée séparément. Le manipulateur disposant de sept articulations pour une tâche de pose à six degrés de liberté, le problème est redondant et son issue doit être fixée par un critère explicite.

La résolution s'effectue, pour chaque point de passage, par une descente de Gauss-Newton amortie sur l'erreur de pose, implémentée en C++ avec la bibliothèque *Pinocchio*. À l'itération courante, l'écart entre la pose atteinte $\mathbf{T}(\mathbf{q})$ et la pose désirée $\mathbf{T}_k^*$ est mesuré dans l'algèbre de Lie de $SE(3)$, via le logarithme du groupe,

$$\mathbf{e}(\mathbf{q}) = \log\left((\mathbf{T}_k^*)^{-1} \mathbf{T}(\mathbf{q})\right) \in \mathbb{R}^6.$$

La Jacobienne géométrique du repère TCP, $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times 7}$, donne le torseur cinématique (vitesses linéaire et angulaire) du TCP produit par une vitesse articulaire. Au premier ordre, elle relie donc un incrément $\Delta\mathbf{q}$ à la variation de l'erreur de pose qu'il induit, $\mathbf{e}(\mathbf{q} + \Delta\mathbf{q}) \approx \mathbf{e}(\mathbf{q}) + \mathbf{J}(\mathbf{q}) \Delta\mathbf{q}$. L'incrément articulaire combine alors une tâche principale, celle de réduire $\mathbf{e}$, ainsi qu'une tâche secondaire confinée au noyau de $\mathbf{J}$, c'est-à-dire aux mouvements articu-

lares qui laissent la pose du TCP inchangée :

$$\Delta \mathbf{q} = \underbrace{-\mathbf{J}^+ \mathbf{e}}_{\text{suivi de pose}} + \underbrace{(\mathbf{I} - \mathbf{J}^+ \mathbf{J}) \left(-k_0 (\mathbf{q} - \mathbf{q}_{\text{ref}}) \right)}_{\text{résolution de la redondance}}, \quad \mathbf{J}^+ = \mathbf{J}^\top (\mathbf{J} \mathbf{J}^\top + \lambda \mathbf{I})^{-1}.$$

Le premier terme, $-\mathbf{J}^+ \mathbf{e}$, est la pseudo-inverse amortie de $\mathbf{J}$ [99, 100] appliquée à l'erreur : l'amortissement λ régularise le voisinage des singularités, où la pseudo-inverse non amortie produirait des pas divergents. Le second terme rapproche la configuration d'une posture de référence $\mathbf{q}_{\text{ref}}$ au moyen de l'objectif $-k_0(\mathbf{q} - \mathbf{q}_{\text{ref}})$, que le projecteur $\mathbf{I} - \mathbf{J}^+ \mathbf{J}$ restreint au noyau de $\mathbf{J}$ [101] : la résolution de la redondance n'altère ainsi pas le suivi de la pose, au premier ordre et hors du voisinage immédiat des singularités. L'incrément résultant est borné en norme ($\|\Delta \mathbf{q}\| \leq \delta_{\text{max}} = 0,2 \text{ rad}$), puis intégré sur la variété des configurations, $\mathbf{q} \leftarrow \mathbf{q} \oplus \Delta \mathbf{q}$, jusqu'à ce que $\|\mathbf{e}\| < \varepsilon = 10^{-5}$ ou que 50 itérations soient atteintes.

La référence $\mathbf{q}_{\text{ref}}$ est prise égale à la configuration d'amorçage (*warm-start*), elle-même propagée le long de l'horizon : la première pose est résolue à partir de la configuration mesurée du robot, et chaque pose suivante à partir de la solution de la précédente. Ce chaînage assure simultanément une résolution cohérente de la redondance en assurant que les $\mathbf{q}_k$ restent voisins et la continuité de la trajectoire articulaire. Un déroulement final ramène les écarts successifs dans $[-\pi, \pi]$ pour éliminer tout saut de 2π . La robustesse numérique repose sur une projection de la rotation cible sur $SO(3)$ et sur le rejet des configurations dégénérées (déterminant ou trace aberrants, échec du logarithme), auquel cas la configuration d'amorçage est conservée.

Le solveur produit ainsi la trajectoire articulaire nominale $\mathbf{q}_{1:H}$, qui est la seule grandeur retenue en aval : elle est publiée, puis ré-échelonnée temporellement (Section 4.6.3) avant d'alimenter le filtre de sécurité.

4.6.3 Réajustement temporel de la trajectoire articulaire

La cinématique inverse fournit une suite de configurations articulaires $\mathbf{q}_1, \dots, \mathbf{q}_H$, mais sans horodatage : le modèle de *Flow Matching* produit un nombre fixe de points de passage dont l'espacement dans l'espace articulaire n'est pas uniforme et varie aussi bien à l'intérieur d'un plan que d'un plan à l'autre. Leur affecter une cadence constante, par exemple un point toutes les 0,1 s, ferait alors varier fortement la vitesse articulaire réelle : élevée entre deux configurations éloignées, faible entre deux configurations voisines, d'où un profil de vitesse irrégulier et saccadé. Le réajustement attribue au contraire à chaque segment une durée proportionnelle à sa longueur articulaire, de sorte que la vitesse reste approximativement constante d'un segment à l'autre.

La longueur d'un segment est mesurée par une norme articulaire pondérée,

$$\ell_i = \left\| \mathbf{W} (\mathbf{q}_{i+1} - \mathbf{q}_i) \right\|, \quad \mathbf{W} = \text{diag}(\mathbf{w}),$$

où les poids $\mathbf{w}$ autorisent à pénaliser différemment les articulations (tous égaux à l'unité dans la configuration retenue, soit la norme euclidienne articulaire). Pour une vitesse articulaire cible v^* (fixée à 0,3 rad/s), la durée de chaque segment et les instants de passage valent

$$\Delta t_i = \frac{\max(\ell_i, \ell_{\min})}{v^*}, \quad t_1 = 0, \quad t_{i+1} = t_i + \Delta t_i,$$

chaque segment étant alors parcouru à la vitesse articulaire constante v^* . Le plancher $\ell_{\min} = f_{\min} L / (H - 1)$, fraction $f_{\min} = 0,02$ de la longueur totale $L = \sum_i \ell_i$ rapportée au nombre de segments, empêche un segment quasi stationnaire de se voir attribuer une durée nulle, qui ferait diverger la vitesse définie ci-dessous.

La trajectoire ré-échelonnée $(\mathbf{q}_i, t_i)$ est publiée sous forme de positions horodatées, sans vitesse explicite. L'exécuteur de trajectoire l'interpole à la fréquence de commande et en publie la vitesse feedforward par différentiation finie :

$$\dot{\mathbf{q}}_{\text{ff}}(t) = \frac{\mathbf{q}_{i+1} - \mathbf{q}_i}{t_{i+1} - t_i}, \quad t \in [t_i, t_{i+1}]. \quad (4.10)$$

Cette vitesse ne provient donc pas d'un second calcul de cinématique inverse, mais de la simple différentiation des positions articulaires aux instants imposés par le réajustement. Par construction de ces instants, sa norme est approximativement constante et proche de v^* malgré des points de passage non équidistants. Elle représente l'intention instantanée de la trajectoire générée par *Flow Matching* et fournit au filtre de sécurité une référence régulière, plutôt qu'une différentiation bruitée d'une consigne de position échantillonnée à cadence fixe.

4.7 Filtre de sécurité réactif SDF-CBF

Le modèle génératif est entraîné sur des démonstrations sans obstacle et exécuté en boucle ouverte : il n'offre aucune garantie vis-à-vis d'un environnement non vu à l'entraînement. Plutôt que de contraindre le processus de génération (au prix d'une dérive distributionnelle et d'un coût de calcul accru, voir le Chapitre 2), la sécurité est déléguée à un filtre réactif agissant sur la vitesse articulaire à la fréquence de commande. Le robot étant protégé sur l'ensemble de ses liens, chaque lien l menacé par un obstacle impose sa propre condition de sécurité ; à chaque cycle de commande, le filtre retient la vitesse la plus proche de l'intention

du modèle qui les respecte toutes :

$$\dot{\mathbf{q}}^* = \arg \min_{\dot{\mathbf{q}} \in \mathbb{R}^7} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|^2 \quad \text{s.c.} \quad \nabla h_l(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h_l(\mathbf{q})), \quad \forall l \in \mathcal{A}. \quad (4.11)$$

où $\mathbf{q} \in \mathbb{R}^7$ est la configuration articulaire mesurée, $\dot{\mathbf{q}}_{\text{base}}$ la vitesse nominale (l'intention du modèle génératif) et $\mathcal{A}$ l'ensemble des liens dont la contrainte est active au cycle courant. Chaque contrainte délimite un demi-espace de vitesses admissibles : la commande retenue est la projection de $\dot{\mathbf{q}}_{\text{base}}$ sur leur intersection. Ce problème comporte trois ingrédients, qui structurent la section. Les fonctions barrière $h_l(\mathbf{q})$, qui mesurent la distance signée entre chaque lien et les points obstacles $\mathbf{p}_i \in \mathbb{R}^3$ de la carte persistante, reposent sur un modèle de distance du robot appris hors ligne (Section 4.7.1), à partir duquel elles sont assemblées et leur gradient ∇h_l obtenu en ligne (Sections 4.7.2 et 4.7.3). Le second membre $b(h_l)$ et la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ sont définis à la Section 4.7.4. Enfin, la Section 4.7.5 résout la projection, décrit le post-traitement qui transforme cette solution en commande exécutable $\dot{\mathbf{q}}_{\text{final}}$, et délimite la portée des garanties obtenues.

4.7.1 Modèle de distance du robot appris hors ligne

Le filtre doit connaître, en quelques millisecondes et pour des milliers de points simultanément, la distance signée entre tout point de l'espace et la surface du robot — ainsi que son gradient. Suivant l'approche *Robot Distance Fields* [75], également exploitée pour l'évitement réactif de collisions dans un cadre de commande à priorité de tâches [77], le SDF de chaque lien l est approximé par une base tensorielle de polynômes de Bernstein dans un repère normalisé propre au lien. La géométrie maillée du lien est d'abord centrée et mise à l'échelle :

$$\bar{\mathbf{x}} = \frac{\mathbf{x} - \mathbf{o}_l}{s_l}, \quad \mathbf{o}_l = \text{centre de la boîte englobante}, \quad s_l = \max_{\mathbf{v} \in \mathcal{V}_l} \|\mathbf{v} - \mathbf{o}_l\|, \quad (4.12)$$

où $\mathcal{V}_l$ est l'ensemble des sommets du maillage : le lien est ainsi contenu dans le cube unitaire $[-1, 1]^3$. Sur ce domaine, le SDF normalisé est une combinaison linéaire de n^3 fonctions de base :

$$\bar{d}_l(\bar{\mathbf{x}}) = \sum_{a,b,c=0}^{n-1} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B_c(u_z) = \Phi(\bar{\mathbf{x}}) \mathbf{w}_l \quad (4.13)$$

$$B_a(u) = \binom{n-1}{a} u^a (1-u)^{n-1-a} \quad (4.14)$$

$$\mathbf{u} = \frac{\bar{\mathbf{x}} + 1}{2} \quad (4.15)$$

et la distance métrique s'en déduit par $d_l = s_l \bar{d}_l$. Chacun des neuf liens protégés (liens 2 à 7, la main et les deux doigts de la pince) utilise $n = 12$ fonctions par axe, soit 1728 coefficients par lien. Trois propriétés motivent ce choix de représentation : le modèle est polynomial, donc $\mathcal{C}^\infty$ et de gradient analytique exact ; il est linéaire en ses paramètres $\mathbf{w}_l$, ce qui rend l'ajustement convexe ; et son évaluation se réduit à des produits tensoriels, massivement parallélisables sur GPU.

L'apprentissage des poids procède en deux étapes par lien. Un jeu d'étiquettes est d'abord généré sur le maillage, suivant le protocole d'échantillonnage de [75] : 8×10^5 points concentrés près de la surface (balayages virtuels, signe déterminé par les normales) et autant de points uniformes dans le cube, chacun étiqueté de sa distance signée exacte ; les étiquettes négatives situées hors de la boîte englobante du lien, artefacts du calcul de signe, sont écartées. La linéarité du modèle permet alors un ajustement par *moindres carrés récursifs* : à chaque itération, un lot de points Φ_k d'étiquettes $\mathbf{d}_k$ met à jour les poids en forme close,

$$\mathbf{K}_k = \mathbf{B}_{k-1} \Phi_k^\top (\mathbf{I} + \Phi_k \mathbf{B}_{k-1} \Phi_k^\top)^{-1} \quad (4.16)$$

$$\mathbf{B}_k = \mathbf{B}_{k-1} - \mathbf{K}_k \Phi_k \mathbf{B}_{k-1} \quad (4.17)$$

$$\mathbf{w}_k = \mathbf{w}_{k-1} + \mathbf{K}_k (\mathbf{d}_k - \Phi_k \mathbf{w}_{k-1}) \quad (4.18)$$

avec $\mathbf{B}_0 = 10^2 \mathbf{I}$ (régularisation de type *ridge*). Cette régression de valeur, qui correspond à l'apprentissage original de [75], fournit une distance précise mais ne contraint pas le gradient : près de la surface, sa direction reste bruitée et sa norme s'écarte de l'unité.

Or le filtre de sécurité consomme précisément ce gradient : ∇h fixe la *direction* de la correction de la projection (4.11) et sa *norme* en fixe l'amplitude. Une seconde étape de raffinement géométrique est donc appliquée. Un jeu de supervision dédié de 20 000 *rayons normaux vérifiés* est construit : des points de surface sont déplacés le long de la normale de leur facette d'un décalage δ tiré aléatoirement, concentré à 75 % dans la coquille critique $[1, 10]$ mm (cohérente avec $d_{\text{safe}} = 15$ mm) et étendu jusqu'à 50 mm. Chaque candidat n'est retenu que s'il est vérifié contre le maillage exact : le point doit être extérieur, sa distance exacte recalculée doit coïncider avec le décalage à 0,25 mm près, et la direction exacte du gradient ne doit pas s'écarter de plus de 5° de la normale de départ — ce qui élimine les rayons corrompus par l'axe médian ou la courbure. Chaque rayon fournit ainsi un triplet exact (point, distance, direction sortante $\hat{\mathbf{n}}$). Les poids sont alors affinés par descente de gradient (Adam, recuit en

cosinus) sur la fonction de coût

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{près}} + \mathcal{L}_{\text{unif}} + \lambda_{\text{ray}} \mathcal{L}_{\text{ray}} + \lambda_{\text{eik}} \mathbb{E}[(\|\nabla \bar{d}\| - 1)^2] \\ & + \mathbb{E}\left[\omega \left(\lambda_{\text{dir}} (1 - \widehat{\nabla \bar{d}} \cdot \hat{\mathbf{n}}) + \lambda_{\text{vec}} \|\nabla \bar{d} - \hat{\mathbf{n}}\|^2 \right)\right] \end{aligned} \quad (4.19)$$

où les trois premiers termes sont les erreurs quadratiques de valeur (points près de la surface, uniformes et rayons, $\lambda_{\text{ray}} = 2$), le terme eikonal ($\lambda_{\text{eik}} = 0,05$) impose $\|\nabla \bar{d}\| = 1$ partout, et les deux derniers alignent la direction du gradient sur la direction exacte des rayons ($\lambda_{\text{dir}} = 0,2$, $\lambda_{\text{vec}} = 0,25$), avec une pondération $\omega = 1 + 4e^{-\bar{d}/\bar{\delta}}$ qui concentre l'effort près de la surface. Le gradient $\nabla \bar{d}$ étant la dérivée analytique d'un polynôme, aucune approximation numérique n'intervient.

Les modèles finaux de chaque lien résultent ainsi de 80 itérations de moindres carrés récursifs, dont le résultat sert d'initialisation au raffinement géométrique de 1500 époques. La précision de distance, la direction du gradient et la propriété eikonale qui en résultent sont caractérisées expérimentalement au Chapitre 5.

4.7.2 Fonction barrière

À l'exécution, chaque point obstacle $\mathbf{p}_i$ est ramené dans le repère normalisé de chaque lien protégé via la cinématique directe, puis évalué par le modèle appris :

$$\mathbf{x}_{l,i}(\mathbf{q}) = \mathbf{T}_l(\mathbf{q})^{-1} \mathbf{p}_i, \quad d_{l,i}(\mathbf{q}) = s_l \Phi\left(\frac{\mathbf{x}_{l,i} - \mathbf{o}_l}{s_l}\right) \mathbf{w}_l.$$

Pour les points situés hors du domaine d'approximation $[-1, 1]^3$, la coordonnée normalisée est bornée au cube et la distance euclidienne au point borné est ajoutée — une correction conservatrice qui prolonge le modèle hors de son domaine d'entraînement. Cette formulation fournit une distance différentiable par rapport à $\mathbf{q}$, interrogeable en parallèle sur GPU pour l'ensemble des points obstacles.

Évaluer cette distance sur la totalité de la carte persistante à chaque cycle serait inutilement coûteux : seuls les points les plus proches du robot peuvent activer la contrainte. La carte est donc d'abord élaguée par un préfiltre sphérique autour des liens protégés, puis les K points de plus faible SDF corps-complet sont conservés :

$$\mathcal{P}_{\text{CBF}} = \text{TopK}_K \left(\min_l d_l(\mathbf{q}, \mathbf{p}_i) \right), \quad K = 100.$$

Cette sélection est mise à jour à une fréquence inférieure à celle de la boucle de commande

(30 Hz contre 100 Hz). L'obsolescence maximale de l'ensemble actif qui en résulte (≈ 33 ms) est prise en compte dans le budget de la marge de sécurité d_{safe} (Section 4.7.5).

Les distances $d_{l,i}$ des points retenus sont alors agrégées, lien par lien, en une barrière scalaire. Le minimum sur les points obstacles y est approché par un soft-min de température α , formulé de manière numériquement stable autour de $m_l = \min_i d_{l,i}$:

$$h_l(\mathbf{q}) = m_l - \alpha \log \left[\sum_{i=1}^N \exp \left(-\frac{d_{l,i} - m_l}{\alpha} \right) \right] - d_{\text{safe}}.$$

Chaque lien protégé possède ainsi sa propre barrière h_l et son propre ensemble admissible $\{\mathbf{q} \mid h_l(\mathbf{q}) \geq 0\}$; l'ensemble sûr du robot en est l'intersection,

$$\mathcal{C} = \bigcap_l \{\mathbf{q} \mid h_l(\mathbf{q}) \geq 0\}.$$

Réduire cette famille à la seule barrière minimale ne contraindrait que le lien instantanément le plus proche et laisserait les autres liens libres de progresser vers leurs propres obstacles, au prix d'une commutation brutale du lien sélectionné dès qu'un second lien devient critique. Le filtre conserve donc les K_a liens de plus faible h_l et traite chacun comme une contrainte distincte; leur assemblage et leur résolution conjointe sont détaillés à la Section 4.7.5. Le soft-min introduit un biais conservateur borné par $\alpha \log N$; en notant h_{soft} la valeur brute issue du soft-min, la valeur compensée $h_{\text{corr}} = h_{\text{soft}} + \alpha \log N$ est utilisée pour l'interprétation des mesures — c'est sous ces deux noms que la barrière apparaît au Chapitre 5. Ce terme étant indépendant de $\mathbf{q}$, le gradient n'est pas affecté.

4.7.3 Gradient analytique de la contrainte

La projection (4.11) repose entièrement sur le gradient $\nabla_{\mathbf{q}} h$, qui en fixe à la fois la direction et l'amplitude. On l'obtient en forme close, par dérivation exacte de chaque maillon de la chaîne, ce qui en garantit le coût borné qu'exige le temps réel. Pour un point obstacle donné, la distance résulte de la composition de trois applications,

$$\mathbf{q} \xrightarrow{\text{cinématique}} \mathbf{x}_{l,i} \xrightarrow{\text{normalisation}} \mathbf{u} \xrightarrow{\text{SDF de Bernstein}} \bar{d}_l,$$

toutes trois dérivables analytiquement. Les différentielles correspondantes sont établies successivement, puis recombinaées par la règle de chaîne et agrégées à travers le soft-min.

Tout d'abord, en ce qui concerne la différentielle cinématique, l'obstacle est fixe dans le repère de base $\mathcal{F}_b$ ($\dot{\mathbf{p}}_i = \mathbf{0}$), mais paraît mobile dans le repère $\mathcal{F}_l$ que les articulations déplacent.

Sa position locale $\mathbf{x}_{l,i} = {}^b\mathbf{R}_l^\top(\mathbf{p}_i - {}^b\mathbf{d}_l)$ vérifie la relation de fermeture $\mathbf{p}_i = {}^b\mathbf{d}_l + {}^b\mathbf{R}_l \mathbf{x}_{l,i}$, où ${}^b\mathbf{R}_l(\mathbf{q})$ et ${}^b\mathbf{d}_l(\mathbf{q})$ sont la rotation et la translation de $\mathbf{T}_l(\mathbf{q})$. En dérivant cette relation par rapport au temps et en annulant $\dot{\mathbf{p}}_i$:

$$\mathbf{0} = {}^b\dot{\mathbf{d}}_l + {}^b\dot{\mathbf{R}}_l \mathbf{x}_{l,i} + {}^b\mathbf{R}_l \dot{\mathbf{x}}_{l,i}.$$

Les blocs linéaire et angulaire de la Jacobienne géométrique du lien, ${}^b\mathbf{J}_{L,l}$ et ${}^b\mathbf{J}_{A,l}$, relient ces dérivées à la vitesse articulaire par ${}^b\dot{\mathbf{d}}_l = {}^b\mathbf{J}_{L,l}\dot{\mathbf{q}}$ et ${}^b\dot{\mathbf{R}}_l = \left[{}^b\mathbf{J}_{A,l}\dot{\mathbf{q}}\right]_{\times} {}^b\mathbf{R}_l$. En les substituant et en isolant le terme de vitesse locale :

$${}^b\mathbf{R}_l \dot{\mathbf{x}}_{l,i} = -{}^b\mathbf{J}_{L,l}\dot{\mathbf{q}} - \left[{}^b\mathbf{J}_{A,l}\dot{\mathbf{q}}\right]_{\times} {}^b\mathbf{R}_l \mathbf{x}_{l,i}.$$

Une multiplication à gauche par ${}^b\mathbf{R}_l^\top$, suivie des identités du produit vectoriel $\mathbf{R}^\top[\mathbf{a}]_{\times}\mathbf{R} = [\mathbf{R}^\top\mathbf{a}]_{\times}$ et $[\mathbf{a}]_{\times}\mathbf{b} = -[\mathbf{b}]_{\times}\mathbf{a}$, isole la Jacobienne recherchée $\mathbf{J}_{x_{l,i}} = \partial\mathbf{x}_{l,i}/\partial\mathbf{q}$, telle que $\dot{\mathbf{x}}_{l,i} = \mathbf{J}_{x_{l,i}}\dot{\mathbf{q}}$:

$$\boxed{\mathbf{J}_{x_{l,i}}(\mathbf{q}) = -{}^b\mathbf{R}_l^\top {}^b\mathbf{J}_{L,l} + \left[\mathbf{x}_{l,i}\right]_{\times} {}^b\mathbf{R}_l^\top {}^b\mathbf{J}_{A,l}}$$

le premier terme traduisant la translation du repère du lien, le second sa rotation. La matrice antisymétrique associée au produit vectoriel y vaut

$$\left[\mathbf{x}_{l,i}\right]_{\times} = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix}, \quad \mathbf{x}_{l,i} = (x_1, x_2, x_3)^\top.$$

La deuxième application, la normalisation qui ramène l'obstacle dans le cube d'apprentissage, $\mathbf{u} = (\mathbf{x}_{l,i} - \mathbf{o}_l)/(2s_l) + \frac{1}{2}\mathbf{1}$, est affine ; sa Jacobienne est la matrice diagonale constante

$$\frac{\partial\mathbf{u}}{\partial\mathbf{x}_{l,i}} = \frac{1}{2s_l} \mathbf{I}_3.$$

La troisième différentielle, le gradient du SDF dans l'espace normalisé, découle de la dérivation exacte de la base tensorielle. La dérivée d'un polynôme de Bernstein de degré $n-1$ se ramène à la base de degré immédiatement inférieur par la règle d'abaissement

$$\frac{dB_a(u)}{du} = (n-1)\left(B_{a-1}^{(n-2)}(u) - B_a^{(n-2)}(u)\right),$$

d'où, en dérivant le produit tensoriel selon chacun des trois axes, le gradient $\nabla_{\mathbf{u}} \bar{d}_l \in \mathbb{R}^3$:

$$\nabla_{\mathbf{u}} \bar{d}_l = \begin{bmatrix} \sum_{a,b,c} w_{abc}^{(l)} B'_a(u_x) B_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B'_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B'_c(u_z) \end{bmatrix}.$$

La composition des trois différentielles, en tenant compte de la mise à l'échelle métrique $d_{l,i} = s_l \bar{d}_l$, fournit le gradient articulaire d'un point obstacle : le facteur s_l se simplifie avec celui de la normalisation, de sorte que le gradient spatial métrique se réduit à $\nabla_{\mathbf{x}} d_{l,i} = \frac{1}{2} \nabla_{\mathbf{u}} \bar{d}_l$. Cette expression vaut pour les points dont la coordonnée normalisée reste dans le cube $[-1, 1]^3$; pour la correction conservatrice appliquée hors de ce domaine (Section 4.7.2), la composante polynomiale s'annule sur les axes bornés au cube et le gradient s'y réduit à la direction radiale unitaire allant du point borné vers l'obstacle, dérivée exacte de la distance euclidienne ajoutée. Les contraintes effectivement actives portant sur des points proches de la surface du robot, c'est néanmoins la branche intérieure qui gouverne la projection. Plutôt que d'assembler explicitement la Jacobienne $\mathbf{J}_{x_{l,i}}$ pour chacun des points — ce qui matérialiserait un tenseur coûteux —, on applique les mêmes identités du produit vectoriel pour réécrire $\nabla_{\mathbf{q}} d_{l,i} = \mathbf{J}_{x_{l,i}}^\top \nabla_{\mathbf{x}} d_{l,i}$ directement dans le repère de base. En notant $\mathbf{g}_{l,i} = {}^b \mathbf{R}_l \nabla_{\mathbf{x}} d_{l,i}$ le gradient spatial exprimé dans $\mathcal{F}_b$ et $\mathbf{r}_{l,i} = \mathbf{p}_i - {}^b \mathbf{d}_l$ le vecteur reliant l'origine du repère du lien à l'obstacle :

$$\nabla_{\mathbf{q}} d_{l,i} = - {}^b \mathbf{J}_{L,l}^\top \mathbf{g}_{l,i} - {}^b \mathbf{J}_{A,l}^\top (\mathbf{r}_{l,i} \times \mathbf{g}_{l,i}).$$

Cette forme, retenue dans l'implémentation, ne requiert que deux produits matrice-vecteur et un produit vectoriel par lien, et évite de construire la Jacobienne point par point.

Il reste à propager ce résultat à travers l'agrégation soft-min, conduite indépendamment pour chaque lien. La dérivation du Log-Sum-Exp fait apparaître les poids du soft-max, ω_i , qui agissent comme un sélecteur lisse du point obstacle dominant :

$$\omega_i = \frac{\exp(-d_{l,i}/\alpha)}{\sum_{j=1}^N \exp(-d_{l,j}/\alpha)} = \text{softmax}_i(-d_{l,i}/\alpha),$$

et le gradient de la barrière d'un lien l est la combinaison convexe correspondante :

$$\nabla_{\mathbf{q}} h_l = \sum_{i=1}^N \omega_i \nabla_{\mathbf{q}} d_{l,i} = \sum_{i=1}^N \omega_i \mathbf{J}_{x_{l,i}}(\mathbf{q})^\top \nabla_{\mathbf{x}} d_{l,i}.$$

Chaque facteur étant la dérivée exacte d'un polynôme ou d'une transformation rigide, ce gradient est obtenu en forme close, sans aucune approximation numérique. Une seule éva-

luation vectorisée produit conjointement les barrières h_l et les gradients $\nabla_{\mathbf{q}} h_l$ de tous les liens protégés, qui alimentent directement les lignes de la contrainte de sécurité résolue à la Section 4.7.5. Le soft-min sur les points lisse la direction de chaque gradient en pondérant les points obstacles les plus proches, au prix du biais déjà mentionné.

Tableau 4.1 Temps d'évaluation conjointe des barrières h_l et de la Jacobienne de contrainte complète ($L \times 7$) pour 100 points obstacles, en fonction du nombre L de liens protégés inclus : gradient analytique en forme close contre différentiation automatique (moyenne $\pm$ écart-type en ms, 100 répétitions), avec l'erreur absolue maximale entre les deux Jacobiennes.

Liens L	Analytique [ms]	Diff. automatique [ms]	Erreur max. gradient
1	5.596 ± 0.413	7.180 ± 0.113	5.960×10^{-8}
2	5.549 ± 0.345	8.010 ± 0.389	2.235×10^{-7}
3	5.432 ± 0.149	8.836 ± 0.457	2.373×10^{-7}
4	5.444 ± 0.106	9.789 ± 0.127	2.373×10^{-7}
5	5.538 ± 0.398	10.349 ± 0.321	2.980×10^{-7}
6	7.644 ± 0.156	12.257 ± 0.517	2.980×10^{-7}
7	7.792 ± 0.315	12.899 ± 0.288	2.980×10^{-7}
8	7.698 ± 0.172	13.479 ± 0.233	2.980×10^{-7}

4.7.4 Contrainte de sécurité et vitesse nominale

Le problème (4.11) requiert encore ses deux entrées : le second membre $b(h)$, qui délimite les vitesses admissibles, puis la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$, centre de l'objectif. Pour la première, la condition CBF classique impose $\nabla h^\top \dot{\mathbf{q}} \geq -\kappa h$. Avec une barrière issue de la perception, le terme linéaire κh présente deux inconvénients pratiques : loin de l'obstacle il autorise des vitesses d'approche arbitrairement grandes, et en violation il exige des vitesses de récupération proportionnelles à la pénétration, amplifiant le bruit de h . La contrainte implémentée sature ces deux régimes :

$$\nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h), \quad b(h) = m_{\text{CBF}} + \begin{cases} -v_{\text{in}} \min(h/h_{\text{act}}, 1), & h > 0, \\ v_{\text{rec}} \min(-h/d_{\text{rec}}, 1), & h \leq 0, \end{cases}$$

la contrainte n'étant évaluée que dans la bande d'activation $h \leq h_{\text{act}}$. Loin de la frontière, la vitesse d'approche normale est ainsi bornée par v_{in} ; à la frontière ($h = 0$), toute vitesse normale entrante est interdite ; en violation, une vitesse de sortie proactive, croissant avec la pénétration jusqu'à v_{rec} à la profondeur de saturation d_{rec} , est exigée. Cette récupération proactive est complétée par l'étape de réparation décrite à la Section 4.7.5, qui réévalue

cette même contrainte saturée sur la commande filtrée et corrige la violation résiduelle que la saturation et le filtrage réintroduisent. Cette formulation appartient à la famille $\nabla h^\top \dot{\mathbf{q}} \geq -\gamma(h)$ avec γ une fonction de classe $\mathcal{K}$ saturée.

La seconde entrée du problème (4.11) est la vitesse que le système *souhaite* exécuter. Celle-ci combine le feedforward retimé $\dot{\mathbf{q}}_{\text{ff}}$ (Section 4.6.3) et une correction de suivi proportionnelle saturée :

$$\dot{\mathbf{q}}_{\text{fb}}^0 = \text{sat}\left(K_p(\mathbf{q}_{\text{nom}} - \mathbf{q}_{\text{cur}}), \dot{q}_{\text{fb,max}}\right).$$

À proximité de la frontière, ce terme de suivi est cependant restreint à sa composante tangentielle d'avancement. En effet, lorsque le filtre dévie le robot de la trajectoire nominale pour des raisons de sécurité, un retour proportionnel complet pointe directement vers la trajectoire — c'est-à-dire contre la correction de sécurité — et les deux termes s'annulent en régime permanent, immobilisant le robot contre la frontière. Avec $\hat{\mathbf{t}} = \dot{\mathbf{q}}_{\text{ff}}/\|\dot{\mathbf{q}}_{\text{ff}}\|$ la direction d'avancement nominale :

$$\dot{\mathbf{q}}_{\text{fb}} = \begin{cases} \max(0, \hat{\mathbf{t}}^\top \dot{\mathbf{q}}_{\text{fb}}^0) \hat{\mathbf{t}}, & h \leq h_{\text{act}}, \\ \dot{\mathbf{q}}_{\text{fb}}^0, & \text{sinon.} \end{cases}$$

La vitesse nominale brute $\dot{\mathbf{q}}_{\text{raw}} = \dot{\mathbf{q}}_{\text{ff}} + \dot{\mathbf{q}}_{\text{fb}}$ est ensuite lissée par un filtre passe-bas de constante τ_{nom} :

$$\gamma_{\text{nom}} = \frac{\Delta t}{\Delta t + \tau_{\text{nom}}}, \quad \dot{\mathbf{q}}_{\text{base},k} = (1 - \gamma_{\text{nom}}) \dot{\mathbf{q}}_{\text{base},k-1} + \gamma_{\text{nom}} \dot{\mathbf{q}}_{\text{raw},k},$$

puis saturée aux limites de vitesse articulaire et annulée sous un seuil de maintien (évitant toute dérive en station). Le résultat $\dot{\mathbf{q}}_{\text{base}}$ est la *vitesse nominale filtrée* : c'est l'entrée du filtre de sécurité, et la référence par rapport à laquelle toutes les interventions de celui-ci sont mesurées au Chapitre 5.

4.7.5 De la vitesse nominale à la commande sécurisée

La commande sécurisée est construite à partir d'une vitesse nominale articulaire, puis modifiée uniquement lorsque la contrainte de sécurité devient active. L'exécuteur de trajectoire fournit d'abord une vitesse feedforward retimée $\dot{\mathbf{q}}_{\text{ff}}$, qui représente l'intention instantanée de la trajectoire générée. À cette vitesse est ajoutée une correction de suivi proportionnelle bornée $\dot{\mathbf{q}}_{\text{fb}}$, calculée à partir de l'erreur entre la consigne nominale courante et la configuration mesurée. La vitesse nominale brute est donc

$$\dot{\mathbf{q}}_{\text{raw}} = \dot{\mathbf{q}}_{\text{ff}} + \dot{\mathbf{q}}_{\text{fb}}.$$

Afin d'éviter les discontinuités dues à l'interpolation temporelle des points de passage, cette vitesse est filtrée par un filtre passe-bas exponentiel de constante de temps τ_{nom} :

$$\mathcal{F}_{\tau_{\text{nom}}}[\dot{\mathbf{q}}_{\text{raw}}]_k = (1 - \gamma_{\text{nom}})\mathcal{F}_{\tau_{\text{nom}}}[\dot{\mathbf{q}}_{\text{raw}}]_{k-1} + \gamma_{\text{nom}}\dot{\mathbf{q}}_{\text{raw},k},$$

avec

$$\gamma_{\text{nom}} = \frac{\Delta t}{\Delta t + \tau_{\text{nom}}}.$$

La vitesse nominale transmise au filtre CBF est ensuite saturée aux limites articulaires :

$$\dot{\mathbf{q}}_{\text{base}} = \text{sat}(\mathcal{F}_{\tau_{\text{nom}}}[\dot{\mathbf{q}}_{\text{raw}}]).$$

La projection CBF calcule la vitesse admissible la plus proche de $\dot{\mathbf{q}}_{\text{base}}$, c'est-à-dire sa projection sur l'intersection des demi-espaces actifs (4.11) :

$$\dot{\mathbf{q}}_{\text{proj}} = \arg \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|^2 \quad \text{s.c.} \quad \nabla h_l^\top \dot{\mathbf{q}} \geq b(h_l), \quad \forall l \in \mathcal{A}.$$

La correction de sécurité $\Delta \dot{\mathbf{q}}_{\text{CBF}} \equiv \dot{\mathbf{q}}_{\text{proj}} - \dot{\mathbf{q}}_{\text{base}}$ n'est pas une entrée du calcul mais la quantité *dérivée* qui mesure l'intervention du filtre : nulle dès que toutes les contraintes sont satisfaites (le filtre est alors transparent) et de norme égale à l'intensité de la correction, c'est elle qui est tracée dans la décomposition des vitesses du Chapitre 5.

La brique élémentaire de la résolution est la projection sur un seul demi-espace, qui admet une forme close. Pour le lien l :

$$\dot{\mathbf{q}} \leftarrow \dot{\mathbf{q}} + \omega \frac{\max(0, b(h_l) - \nabla h_l^\top \dot{\mathbf{q}})}{\|\nabla h_l\|^2 + \varepsilon} \nabla h_l.$$

Si la contrainte du lien est déjà respectée, le terme $\max(0, \cdot)$ s'annule et la vitesse reste inchangée ; sinon, elle est déplacée le long de ∇h_l juste assez pour ramener le lien sur sa frontière de sécurité, le facteur de relaxation ω pondérant ce pas. Lorsqu'un *seul* lien est actif, une application de cette mise à jour résout exactement (4.11), et $\Delta \dot{\mathbf{q}}_{\text{CBF}}$ se réduit à cette expression close.

Avec plusieurs liens actifs, l'intersection des demi-espaces n'admet plus de forme fermée. Projeter sur le demi-espace d'un lien pouvant re-violer celui d'un lien déjà traité, on balaie $\mathcal{A}$ à plusieurs reprises (méthode de Gauss–Seidel, ou *projections sur ensembles convexes*) : gradients figés à la configuration courante, la suite des projections converge vers une vitesse qui satisfait toutes les contraintes simultanément. Comme un lien déjà satisfait laisse $\dot{\mathbf{q}}$

inchangé, compléter $\mathcal{A}$ à un effectif fixe est sans effet sur la solution : la sélection $top-K_a$ et la boucle de N_{it} balayages ont une forme constante, capturée une seule fois dans un graphe CUDA puis exécutée intégralement sur le GPU à chaque cycle de commande, sans masquage dynamique ni synchronisation GPU–CPU. C’est cette structure à forme fixe qui rend abordable, à la fréquence de commande, la contrainte conjointe de tous les liens protégés du bras et de l’effecteur.

La vitesse projetée est ensuite saturée aux limites articulaires :

$$\dot{\mathbf{q}}_{\text{proj,sat}} = \text{sat}(\dot{\mathbf{q}}_{\text{proj}}).$$

Cette vitesse saturée est filtrée par un second filtre passe-bas exponentiel de constante de temps τ_{safe} :

$$\dot{\mathbf{q}}_{f,k} = (1 - \gamma_{\text{safe}})\dot{\mathbf{q}}_{f,k-1} + \gamma_{\text{safe}}\dot{\mathbf{q}}_{\text{proj,sat},k},$$

avec

$$\gamma_{\text{safe}} = \frac{\Delta t}{\Delta t + \tau_{\text{safe}}}.$$

L’ordre saturation puis filtrage évite que le filtre mémorise une vitesse physiquement irréalisable.

La saturation et le filtrage peuvent toutefois réintroduire une légère violation de la contrainte CBF. La contrainte est donc réévaluée sur la vitesse filtrée, avec le même second membre saturé $b(h)$ que la projection principale (Section 4.7.4) :

$$c_f = \nabla h^\top \dot{\mathbf{q}}_f - b(h).$$

Si $c_f < 0$ dans la bande d’activation, une réparation finale est appliquée :

$$\Delta \dot{\mathbf{q}}_{\text{rep}} = \frac{\max(0, -c_f)}{\|\nabla h\|^2 + \varepsilon} \nabla h.$$

Cette réparation est une projection corrective sur la même contrainte CBF linéarisée : elle réutilise la valeur de barrière h , le gradient ∇h et la borne $b(h)$ de la projection principale. La commande finale est ainsi certifiée contre exactement la même condition de sécurité que la solution du solveur, et non contre une seconde condition distincte.

Deux garde-fous bornent cette réparation. D’une part, sa norme est plafonnée : à Δv_{max} tant que la barrière est positive, et à un plafond distinct $\Delta v_{\text{max}}^{\text{rec}}$, plus permissif, lorsque $h < 0$ et qu’une sortie rapide de la zone de violation devient prioritaire. Ce plafond couvre notamment le scénario où la carte persistante fait apparaître de nouveaux points obstacles déjà très

proches du robot, voire déjà dans la zone de violation : la violation instantanée $-c_f$ peut alors être grande, et une réparation non bornée se traduirait par une commande de vitesse violente en un seul cycle. D'autre part, la réparation est modulée par un seuil de fiabilité du gradient : son amplitude croît linéairement de zéro, lorsque $\|\nabla h\| = g_{\min}$, jusqu'à sa pleine valeur à $3g_{\min}$. La direction $\nabla h/\|\nabla h\|$ n'est en effet fiable que si la norme du gradient est significative : face à un obstacle mince, les gradients des points situés de part et d'autre du lien se compensent dans l'agrégation soft-min et $\|\nabla h\|$ s'effondre ; une réparation de pleine amplitude pousserait alors le bras dans une direction dépourvue de sens géométrique. Le filtre s'en remet dans ce cas à la vitesse issue de la projection pour le cycle courant, la violation résiduelle étant résorbée aux cycles suivants, dès que la géométrie redevient non dégénérée. La même limite $\Delta v_{\max}$ s'applique enfin à chaque pas de projection de la boucle de balayage elle-même : lorsque cette borne est atteinte, la solution retournée n'est plus la projection exacte de (4.11), mais une solution dont la correction par cycle est bornée, le résidu étant repris par la réévaluation finale ci-dessus puis par les cycles suivants.

La vitesse finale envoyée à la commande est donc

$$\dot{\mathbf{q}}_{\text{final}} = \text{sat}(\dot{\mathbf{q}}_f + \Delta \dot{\mathbf{q}}_{\text{rep}}).$$

Le contrôleur bas niveau du robot étant un contrôleur de *vitesse* articulaire, cette vitesse certifiée constitue directement la commande envoyée à l'actionneur :

$$\dot{\mathbf{q}}_{\text{cmd},k} = \dot{\mathbf{q}}_{\text{final},k},$$

sans étape d'intégration en une consigne de position. La commande de sécurité reste ainsi exactement la vitesse que la contrainte CBF a certifiée au cycle courant : aucune dynamique d'intégration ne s'interpose entre la projection et l'actionneur, et la garantie en temps discret se réduit au seul maintien de cette vitesse pendant une période de commande (bloqueur d'ordre zéro), déjà comptabilisé dans la marge d_{safe} (Section 4.7.5). Une référence de position intégrée $\mathbf{q}_{\text{ref},k} = \mathbf{q}_{\text{ref},k-1} + \dot{\mathbf{q}}_{\text{final},k} \Delta t$ est néanmoins maintenue en parallèle, à des fins de supervision et de visualisation, mais elle n'est pas l'entrée du contrôleur. Lorsque la contrainte est inactive et que la projection ne modifie pas la vitesse nominale, $\dot{\mathbf{q}}_{\text{final}}$ se réduit à la vitesse nominale lissée, transmise telle quelle.

Au-delà du solveur lui-même, il reste à délimiter ce que le filtre garantit réellement. L'invariance avant théorique de l'ensemble $\mathcal{C}$ suppose une dynamique en temps continu, un SDF exact, des obstacles correctement observés et immobiles, et un suivi parfait de la commande par le contrôleur bas niveau. L'implémentation s'écarte de ces hypothèses sur quatre points

quantifiables : la discrétisation de la commande (100 Hz), l’obsolescence de l’ensemble des points critiques (mise à jour à 30 Hz), l’erreur d’approximation du SDF de Bernstein (caractérisée au Chapitre 5) et le biais conservateur du soft-min ($\alpha \log N$). S’y ajoute le cas dégénéré du gradient effondré face aux obstacles minces, où la réparation finale est volontairement atténuée (seuil $g_{\min}$, Section 4.7.5) et la résorption de la violation reportée aux cycles suivants. Le système constitue donc un filtrage réactif de sécurité dont la garantie pratique repose sur le dimensionnement de la marge d_{safe} pour absorber la somme de ces termes, validé expérimentalement au Chapitre 5.

Tableau 4.2 Paramètres du filtre de sécurité SDF-CBF.

Symbole	Description	Valeur
d_{safe}	marge de sécurité du SDF	15 mm
α	température du soft-min	0,002
K	points critiques sélectionnés	100
h_{act}	bande d’activation de la contrainte	40 mm
v_{in}	vitesse d’approche normale maximale ($\dot{h}$)	0,30 m/s
v_{rec}	vitesse de sortie exigée en violation ($\dot{h}$)	0,30 m/s
d_{rec}	profondeur de saturation en récupération	15 mm
Δv_{max}	correction de projection et de réparation maximale	0,30 rad/s
$\Delta v_{\text{max}}^{\text{rec}}$	réparation maximale en violation ($h < 0$)	0,85 rad/s
$g_{\min}$	seuil de fiabilité du gradient de réparation	0,03
m_{CBF}	marge numérique de la contrainte	0,001 m/s
$K_p, \dot{q}_{\text{fb,max}}$	suivi proportionnel et saturation	5, 0,5 rad/s
$\tau_{\text{nom}}, \tau_{\text{safe}}$	constantes des filtres passe-bas	0,08 s, 0,08 s
Δq_{lead}	avance maximale de la référence	0,05 rad
$K_a, N_{\text{it}}, \omega$	liens contraints, passes POCS, relaxation	9, 6, 1,0

CHAPITRE 5 RÉSULTATS ET DISCUSSION

Présentation avec CBF et sans CBF

Présentation de la détection avec SAM 3

Listes des figures pour les résultats

- Trajectoire de la pointe de la fourchette dans le nuage persistant map ;
 1. Nuage de points de la scène ;
 2. Trajectoire sans CBF
 3. Trajectoire avec CBF
 4. Cible.
- Évolution de $h(q)$ en fonction du temps ;
 1. h_{corr} ;
 2. ligne zéro.
 3. zones où le CBF est actif (en rouge par exemple).
- Distance obstacle-robot en fonction du temps
- Décomposition des vitesses en fonction du temps
 1. $|dq_{ff}|$;
 2. $|dq_{feedback}|$;
 3. $|dq_{base}|$
 4. $|dq_{cbf_delta}|$
 5. $|dq_{escape}|$
 6. $|dq_{safe}|$.
 7. Objectif : Expliquer le comportement du système et montré quand le FLOW Matching dirige le robot seul, quand le CBF intervient, quand l'échappement est nécessaire, si la vitesse corrigée devient trop agressive, etc.
- Effet du filtre passe bas
 1. Vitesse avant CBF brute
 2. Vitesse avant CBF filtrée
- Temps de calcul du pipeline. Faire un boxplot (ou un tableau) sur :
 1. Temps de détection avec SAM3

2. Temps de suivi avec SAM2
 3. Temps de mise à jour persistent map
 4. Temps de génération de trajectoire avec Flow Matching
 5. Temps de calcul de la cinématique inverse
 6. Temps de calcul des points critiques
 7. Temps de calcul de la correction de trajectoire
 8. Temps total du pipeline
- Résistance faces aux occlusions et pertes de détection.
 1. Détection dispo / indispo dans le temps
 2. Images présentant pertes de détection puis redétection
 - NE PAS OUBLIER LA DÉTECTION INITIALE DES FORK TEETH
 - Cas d'échec (comme la figure de Félix).

5.1 Protocol expérimental

5.1.1 Description des scénarios testés

Pick and place

Robot assisted feeding

5.1.2 Métriques utilisées

À compléter.

5.2 validation du modèle géométrique de sécurité

Cette section valide le modèle géométrique au cœur du filtre de sécurité : la SDF du robot approximée par polynômes de Bernstein (section 4.7.1). Trois propriétés sont évaluées par rapport à la vérité terrain calculée sur les maillages du robot : la précision de la distance, la qualité de la direction du gradient et le temps de calcul.

Le protocole est le suivant : pour chacun des six corps protégés, des rayons sont lancés depuis la surface le long de la normale sortante (40 rayons par corps), et la SDF est évaluée en 25 échantillons par rayon, entre 1 mm et 50 mm de la surface. Seuls les rayons *propres* sont conservés, c'est-à-dire ceux dont la distance exacte à l'union des corps coïncide avec l'avancée le long de la normale, ce qui garantit que l'axe des abscisses des figures correspond bien à la

distance réelle à la surface du robot. Les échantillons situés hors du domaine d’entraînement du modèle d’un corps (7,7 % des points, principalement au-delà de 35 mm) sont exclus, car le modèle y retourne une borne géométrique plutôt que la SDF apprise. Au total, 5541 points extérieurs vérifiés sont retenus.

5.2.1 Précision du SDF du robot

La figure 5.2 compare la distance prédite à la distance exacte. L’erreur absolue moyenne est de 1,96 mm avec un biais négligeable ($-0,02$ mm) et une corrélation $R = 0,9869$. La distribution des erreurs signées est centrée et symétrique, avec un 99^e centile de l’erreur absolue de 6,8 mm.

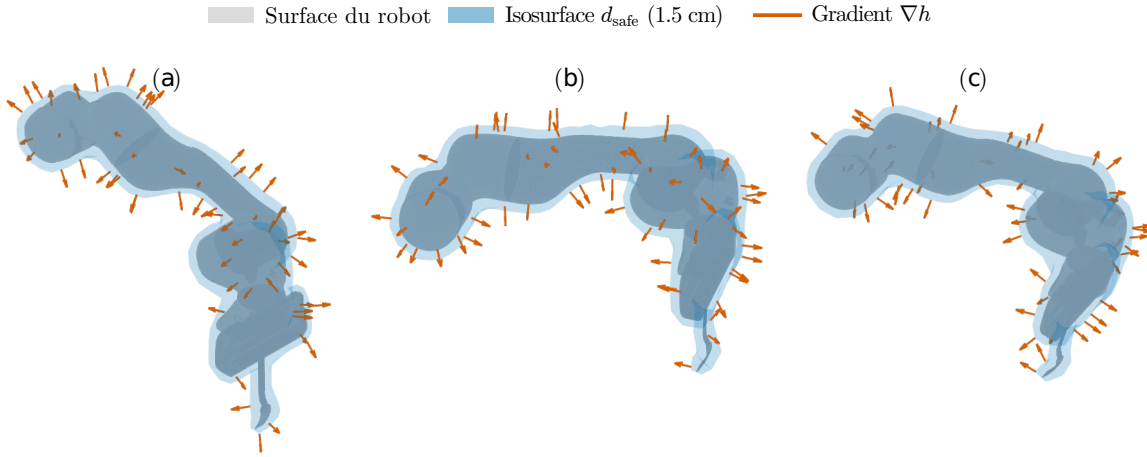


Figure 5.1 Précision de la distance du SDF de Bernstein. (a) Profil de distance le long des normales à la surface du robot ; (b) distribution de l’erreur signée par rapport à la distance exacte calculée sur les maillages.

La figure 5.8(a) présente l’erreur angulaire entre le gradient du modèle et la direction exacte (vecteur vers le point de surface le plus proche). L’erreur médiane globale est de $5,4^\circ$, et reste sous 10° pour la grande majorité des points dès 5 mm de la surface. La figure 5.8(b) montre la norme du gradient : une SDF exacte satisfait la propriété eikonale $\|\nabla h\| = 1$, et le modèle s’en approche au-delà de 10 mm, avec un affaissement vers 0,7 tout près de la surface, où le polynôme lisse le passage par zéro.

Ces résultats justifient la marge de sécurité $d_{\text{safe}} = 15$ mm retenue au chapitre précédent : l’erreur de distance au 99^e centile (6,8 mm) reste largement inférieure à la marge, de sorte qu’une sous-estimation de la distance par le modèle ne suffit pas à amener le robot au contact. De plus, à d_{safe} , la direction du gradient (erreur médiane $\approx 6^\circ$) et sa norme (≈ 1) sont déjà

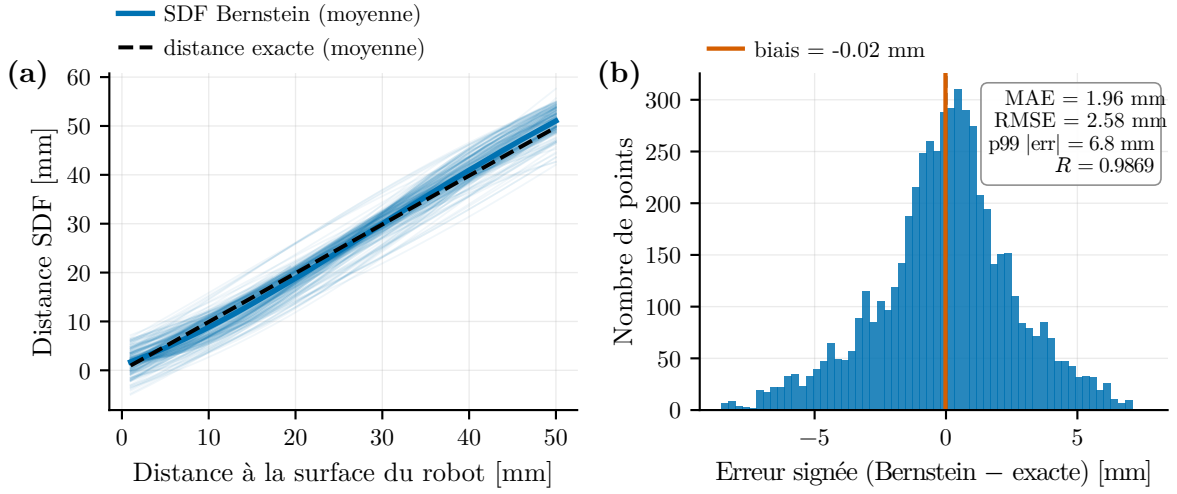


Figure 5.2 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).

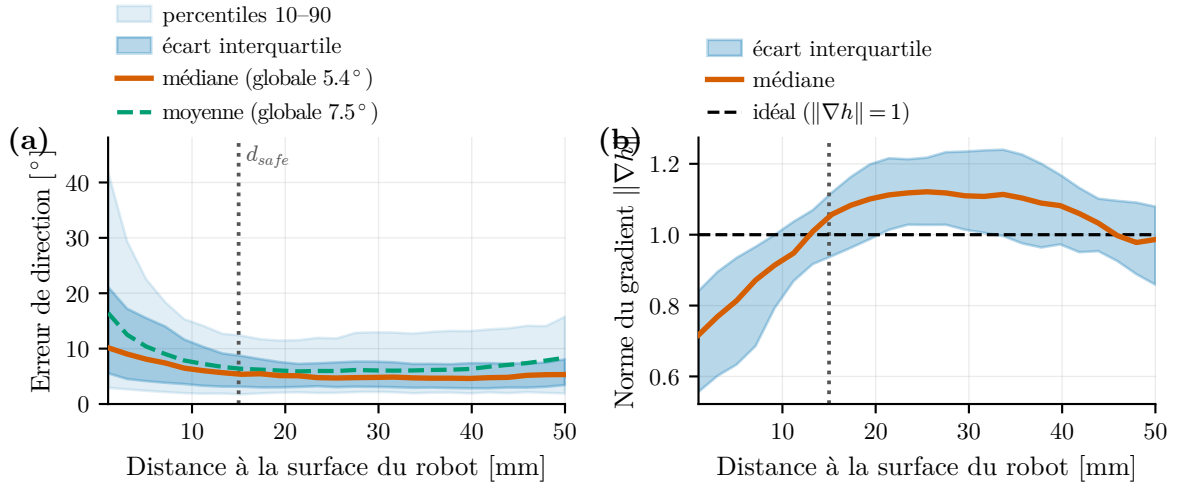


Figure 5.3 Trajectoire multimodale générée par *Flow Matching*.

bien conditionnées, ce qui assure que la contrainte Constraint Barrier Function (CBF) pousse le robot dans la bonne direction au moment où le filtre s'active.

5.2.2 Temps d'évaluation du SDF et du gradient

La figure 5.4 compare le temps d'évaluation conjointe de la distance et du gradient entre le modèle de Bernstein (analytique, sur GPU) et la vérité terrain par requête de proximité sur les maillages (**trimesh**, sur CPU). Le temps du modèle forme un plateau d'environ 5 ms jusqu'à quelques milliers de points : à ces tailles, le coût est dominé par des frais fixes par appel (cinématique directe, $\approx 2,5$ ms, lancement des noyaux GPU, évaluation du gradient et transferts mémoire), le coût par point étant négligeable tant que le GPU n'est pas saturé. La croissance ne devient visible qu'au-delà d'environ 4096 points, soit bien après le point d'opération du filtre ($K = 100$ points critiques). Au point d'opération, l'évaluation reste donc largement en deçà du budget de 13,3 ms imposé par la boucle de commande à 75 Hz, tandis que la requête sur maillage croît linéairement avec le nombre de points et dépasse ce budget d'un à deux ordres de grandeur, ce qui exclut son utilisation en ligne.

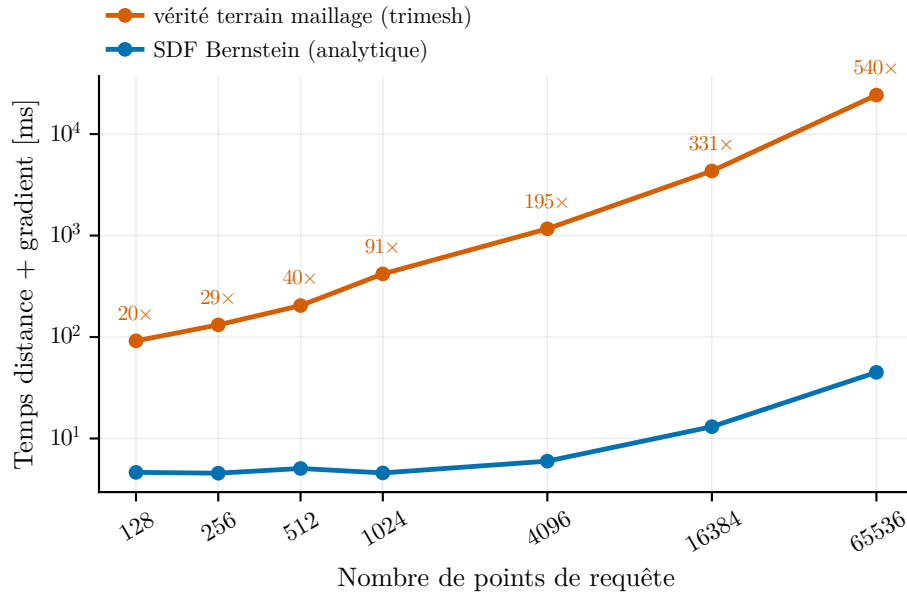


Figure 5.4 Temps de calcul de la distance et du gradient : modèle de Bernstein (GPU) contre vérité terrain sur maillages (CPU), en fonction du nombre de points de requête.

5.2.3 Visualisation des points critiques sélectionnés

Enveloppe de sécurité correspondant à l'isosurface ($d_{\text{robot}} = 1,5$ cm).

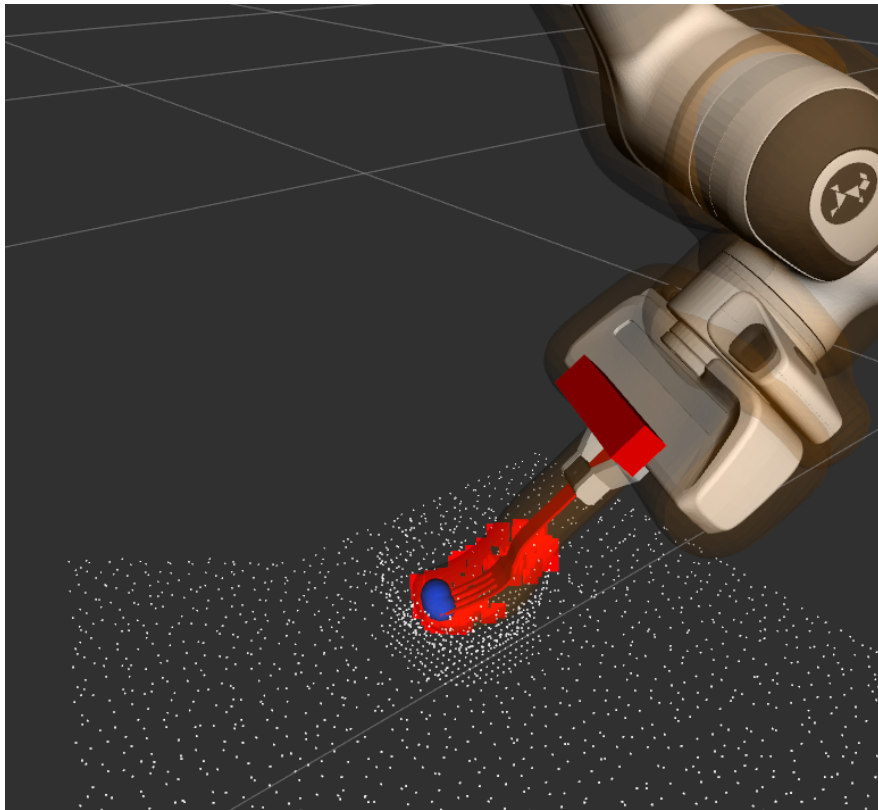


Figure 5.5 Sélection des points les plus proches pour contraindre le CBF

5.3 Qualité de la génération nominale

5.3.1 Trajectoire générée sans obstacle critique

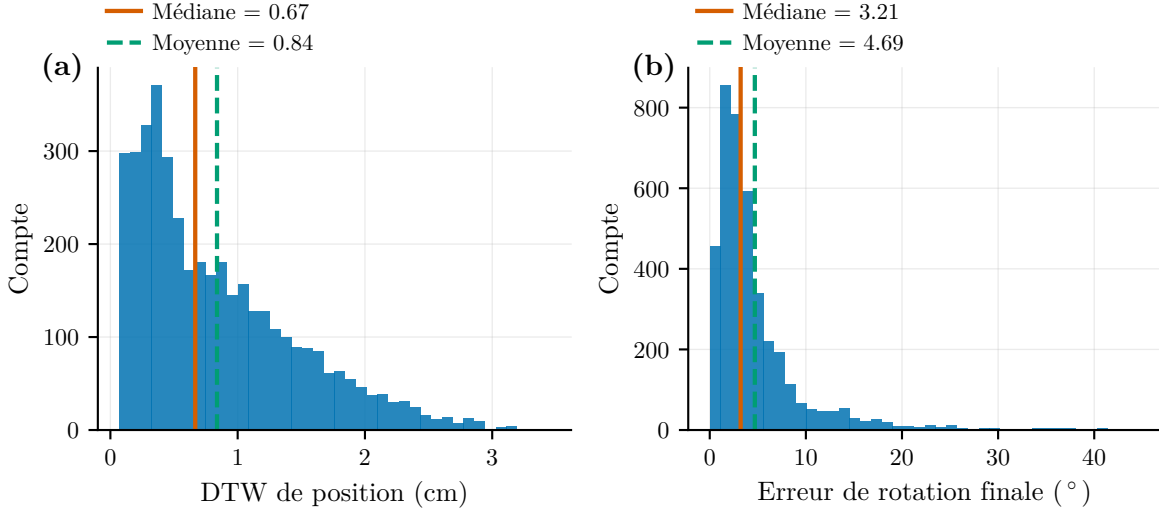


Figure 5.6 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).

5.3.2 Cohérence de la trajectoire avec la cible

À compléter.

À compléter.

À compléter.

À compléter.

5.4 Sécurité et évitement d'obstacle

5.4.1 Comparaison des trajectoires avec et sans SDF-CBF

5.4.2 Évolution de $h(q)$ et de la contrainte de sécurité

La Figure 5.12 présente l'évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} au cours d'un essai représentatif. Les bandes vertes marquent les cycles où la projection corrige la vitesse ($\Delta \dot{\mathbf{q}}_{\text{CBF}} \neq 0$) ; les bandes jaunes, ceux où seule la réparation intervient — la vitesse

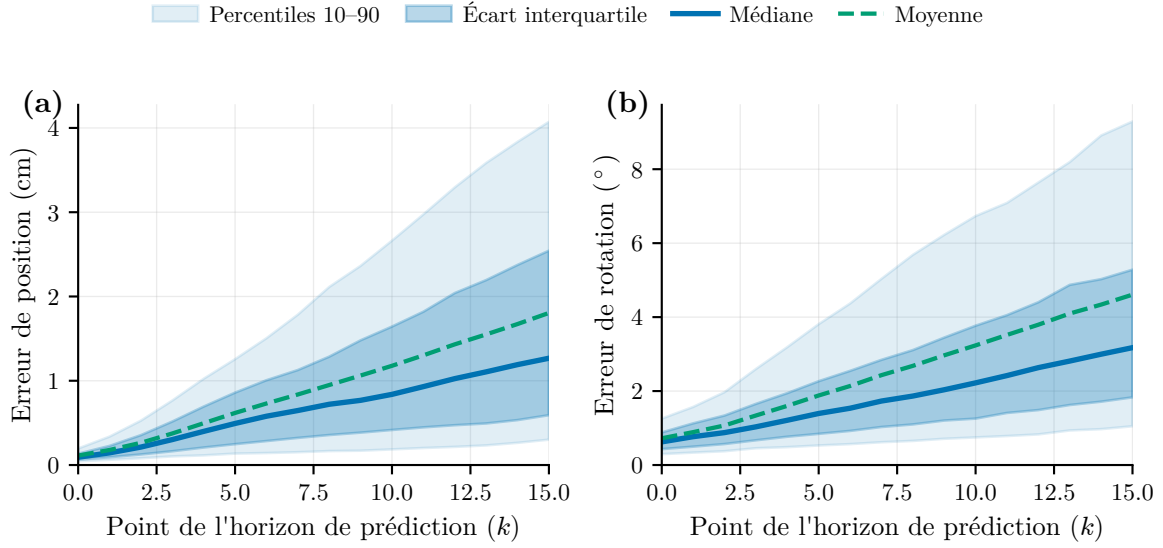


Figure 5.7 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

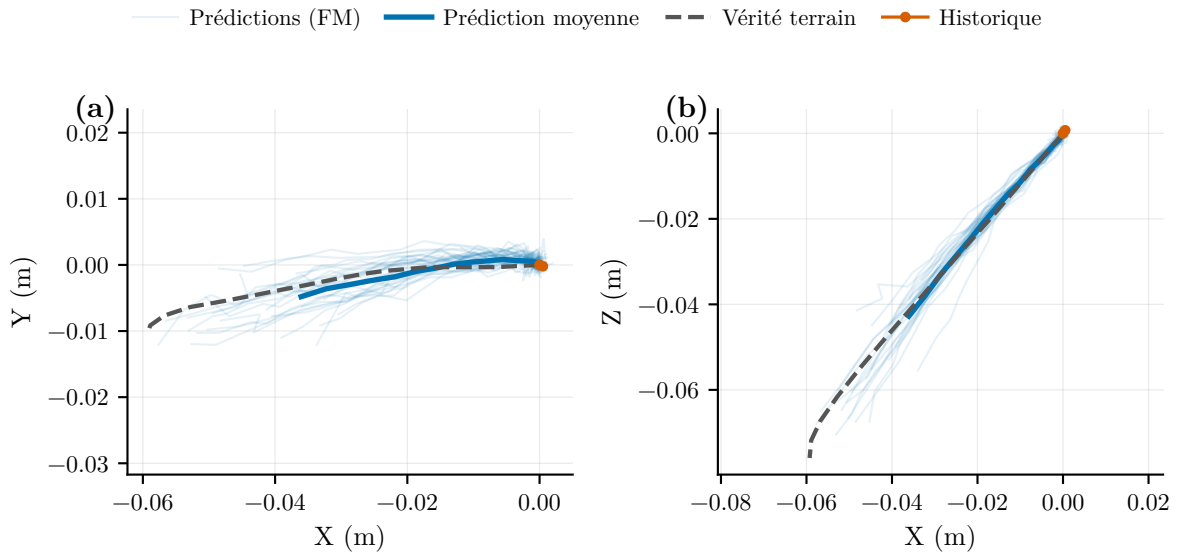


Figure 5.8 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

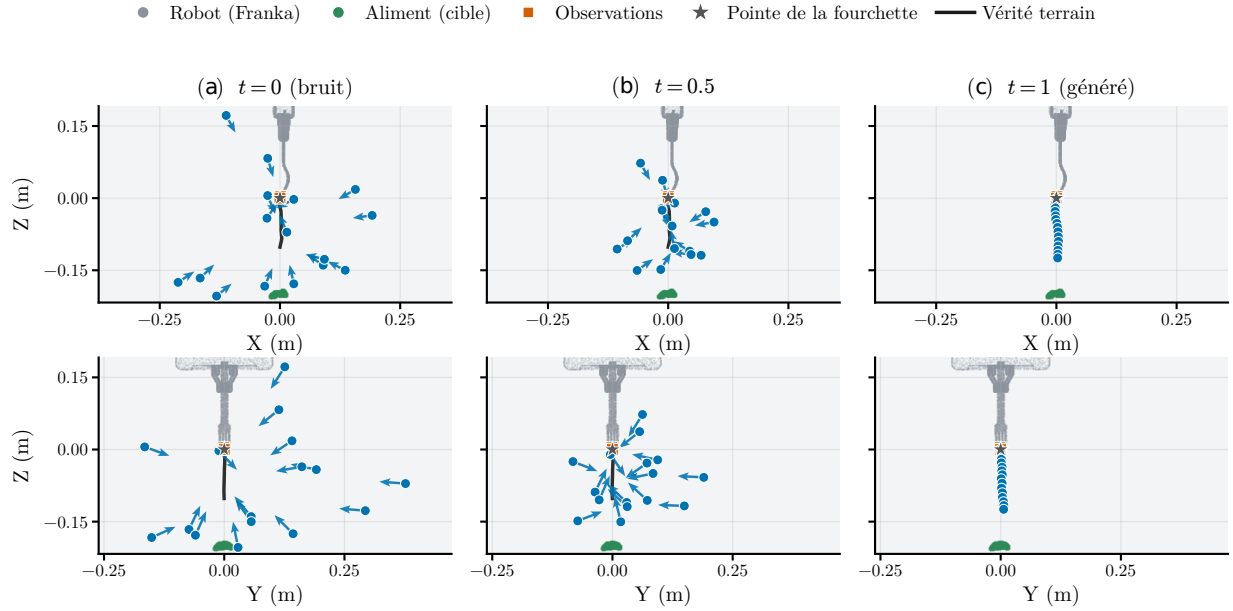


Figure 5.9 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

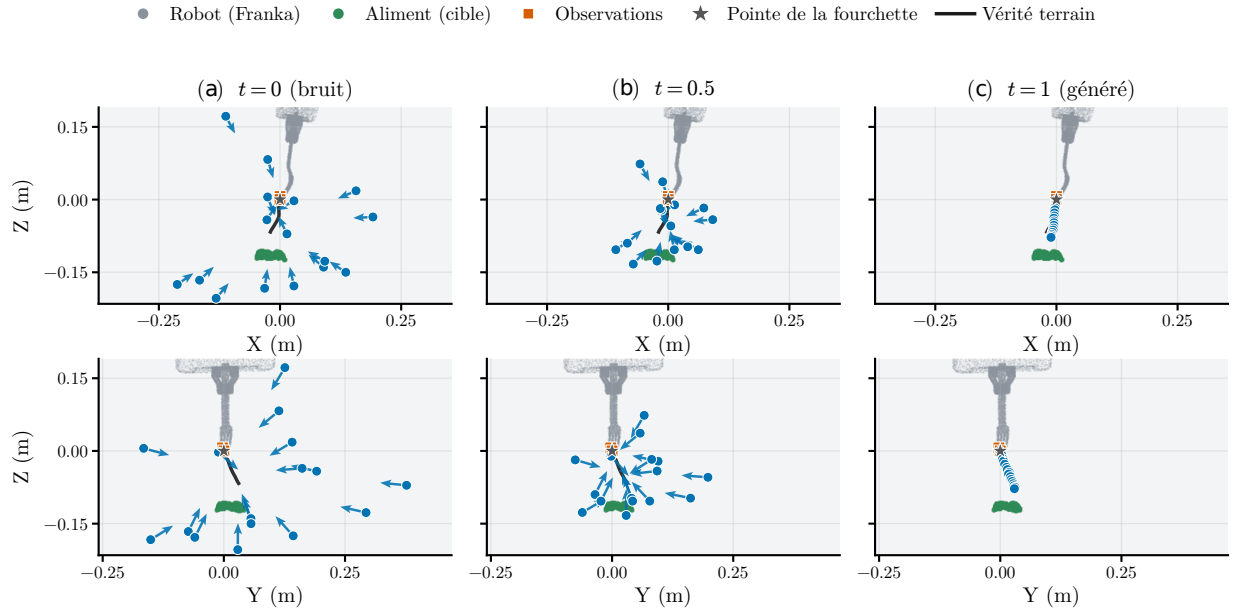


Figure 5.10 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

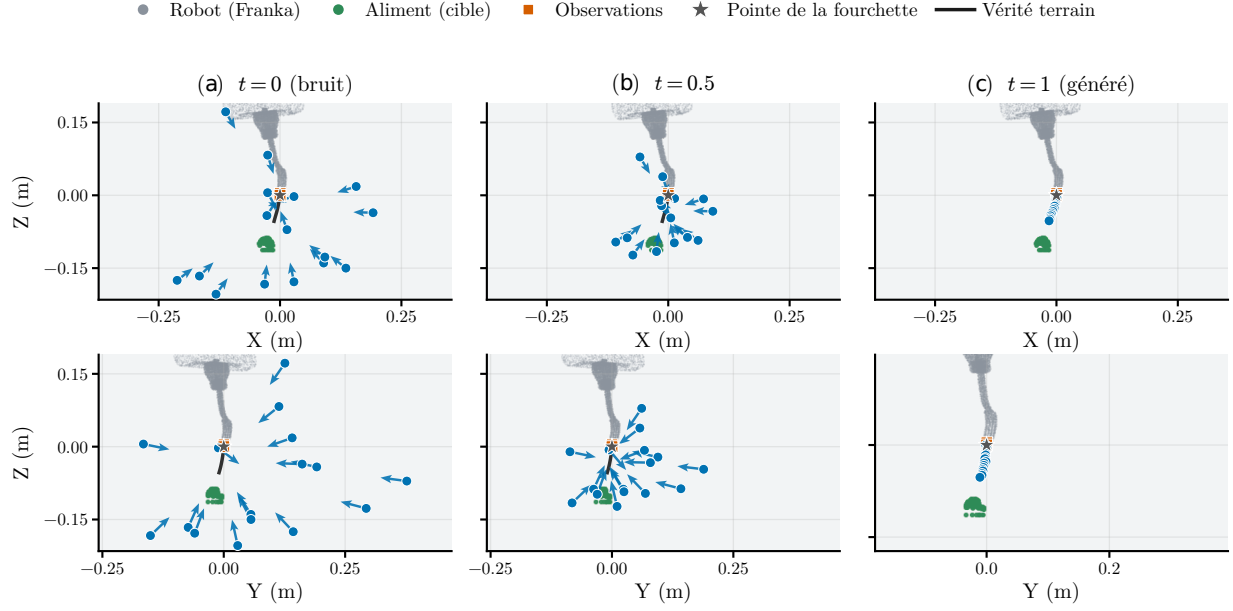


Figure 5.11 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).

instantanée satisfait la contrainte, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la revérification et est corrigée par $\Delta \dot{\mathbf{q}}_{\text{rep}}$. Après la phase d'approche, le système opère en régime de suivi de frontière : h_{corr} se stabilise à quelques millimètres de la frontière pendant que le robot progresse tangentiellement le long des surfaces vers la cible, conformément au comportement attendu de la projection préservant la composante tangentielle (Section 4.7.5).

La Figure 5.13 compare la valeur de la contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après filtrage et réparation). La courbe nominale plonge sous zéro dès que le robot opère à proximité des obstacles : laissée seule, la trajectoire générée par *Flow Matching* violerait continuellement la condition de sécurité. La courbe finale ne descend jamais sous $-m_{\text{CBF}}$ (pire valeur -5×10^{-4}) : le résidu introduit par le filtrage et les saturations est entièrement absorbé par la marge numérique, de sorte que la condition CBF proprement dite, $\nabla h^\top \dot{\mathbf{q}} + \kappa_{\text{eff}} h \geq 0$, est satisfaite à chaque cycle de l'essai. L'étape de réparation (Section 4.7.5) est loin d'être marginale dans ce résultat : elle intervient sur la quasi-totalité des cycles où la projection est active, le filtre de sortie érodant systématiquement une partie de la correction au sein du même cycle. L'écart entre les deux courbes constitue la mesure directe de l'intervention du filtre de sécurité.

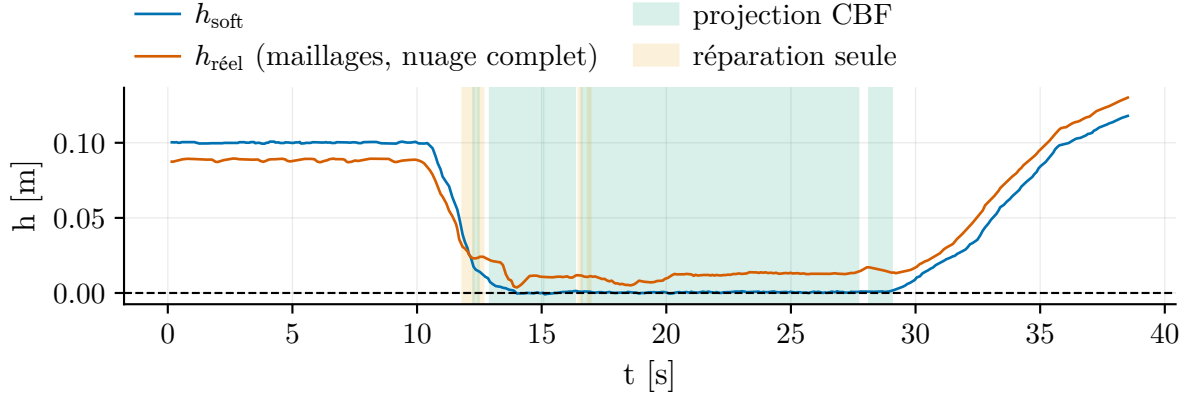


Figure 5.12 Évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} pour une tâche en présence d'obstacles ; les bandes vertes indiquent les cycles où la projection corrige la vitesse, les bandes jaunes ceux où seule la réparation intervient.

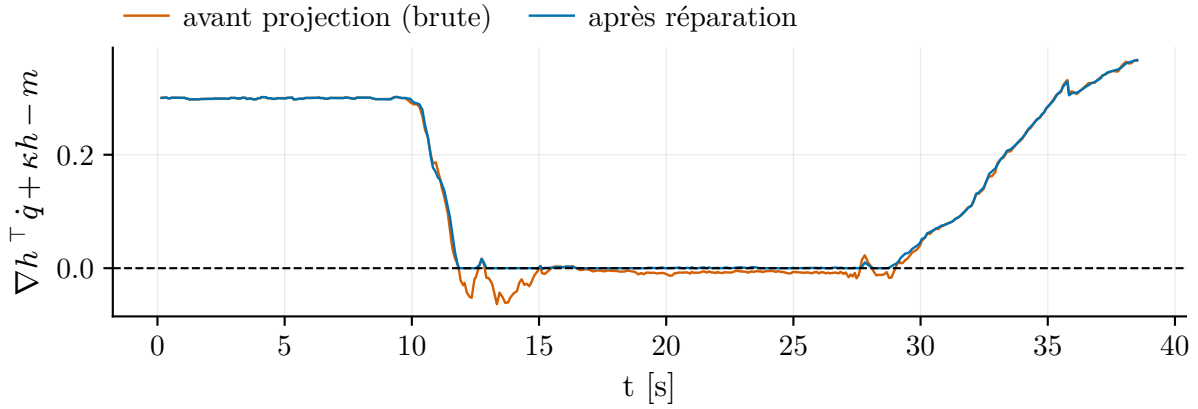


Figure 5.13 Contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après réparation). Les valeurs négatives de la courbe nominale correspondent aux violations qu'aurait commises la trajectoire générée sans filtre.

5.4.3 Déviation entre trajectoire nominale et trajectoire sécurisée

À compléter.

5.5 Analyse des vitesses et de la fluidité

5.5.1 Décomposition des vitesses articulaires

La Figure 5.14 déroule la chaîne complète des Sections 4.7.4 à 4.7.5. Le panneau (a) montre la construction de la vitesse nominale : le feedforward retimé $\dot{\mathbf{q}}_{\text{ff}}$, de norme constante par construction, la correction de suivi $\dot{\mathbf{q}}_{\text{fb}}$, et la vitesse nominale filtrée $\dot{\mathbf{q}}_{\text{base}}$ qui en résulte. Le panneau (b) montre ce que le filtre de sécurité en fait : les deux corrections, projection $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ et réparation $\Delta\dot{\mathbf{q}}_{\text{rep}}$, et la vitesse effectivement commandée $\dot{\mathbf{q}}_{\text{final}}$, superposées à $\dot{\mathbf{q}}_{\text{base}}$ comme référence commune aux deux panneaux. La réparation n'étant pas journalisée par le nœud, $\Delta\dot{\mathbf{q}}_{\text{rep}}$ est reconstituée en rejouant le filtre passe-bas de sortie sur les signaux enregistrés, reconstruction validée par un résidu médian de 10^{-4} rad/s sur les cycles sans réparation. Les normes sont exprimées dans l'espace articulaire (rad/s) ; la vitesse cartésienne de l'outil s'en déduit par la jacobienne et varie avec la configuration du bras.

Trois observations ressortent. Premièrement, $\|\dot{\mathbf{q}}_{\text{ff}}\|$ est constante à 0,25 rad/s sur toute la tâche : le retiming à vitesse articulaire constante (Section 4.6.3) remplit son rôle et fournit au filtre une référence régulière. Deuxièmement, la correction de projection est strictement nulle hors des zones d'activation : le filtre est transparent dès que la contrainte est satisfaite, ce qui confirme le mécanisme de transmission directe. On remarque d'ailleurs des épisodes où $\Delta\dot{\mathbf{q}}_{\text{rep}}$ est active alors que $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ demeure nulle (vers 5 s et 22 s) : la barrière y effleure la frontière sans la presser — la projection juge la vitesse instantanée admissible, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la revérification et est corrigée par la réparation. Troisièmement, la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ présente un bruit important dont l'analyse spectrale situe la puissance dominante à 12 Hz, soit exactement la fréquence de régénération du planificateur : chaque nouvelle trajectoire générée déplace légèrement la consigne, et le gain de suivi K_p transforme ces sauts en impulsions de vitesse.

5.5.2 Action du filtre de sécurité sur la vitesse commandée

La Figure 5.15 superpose la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ (ce que le robot ferait en l'absence d'obstacle) et la vitesse commandée $\dot{\mathbf{q}}_{\text{final}}$. Sur les cycles où le filtre intervient effectivement (projection ou réparation), la surface entre les deux courbes est teintée selon le sens de l'intervention : freinage lorsque le filtre retire de la vitesse, poussée lorsqu'il en ajoute (récupération hors de

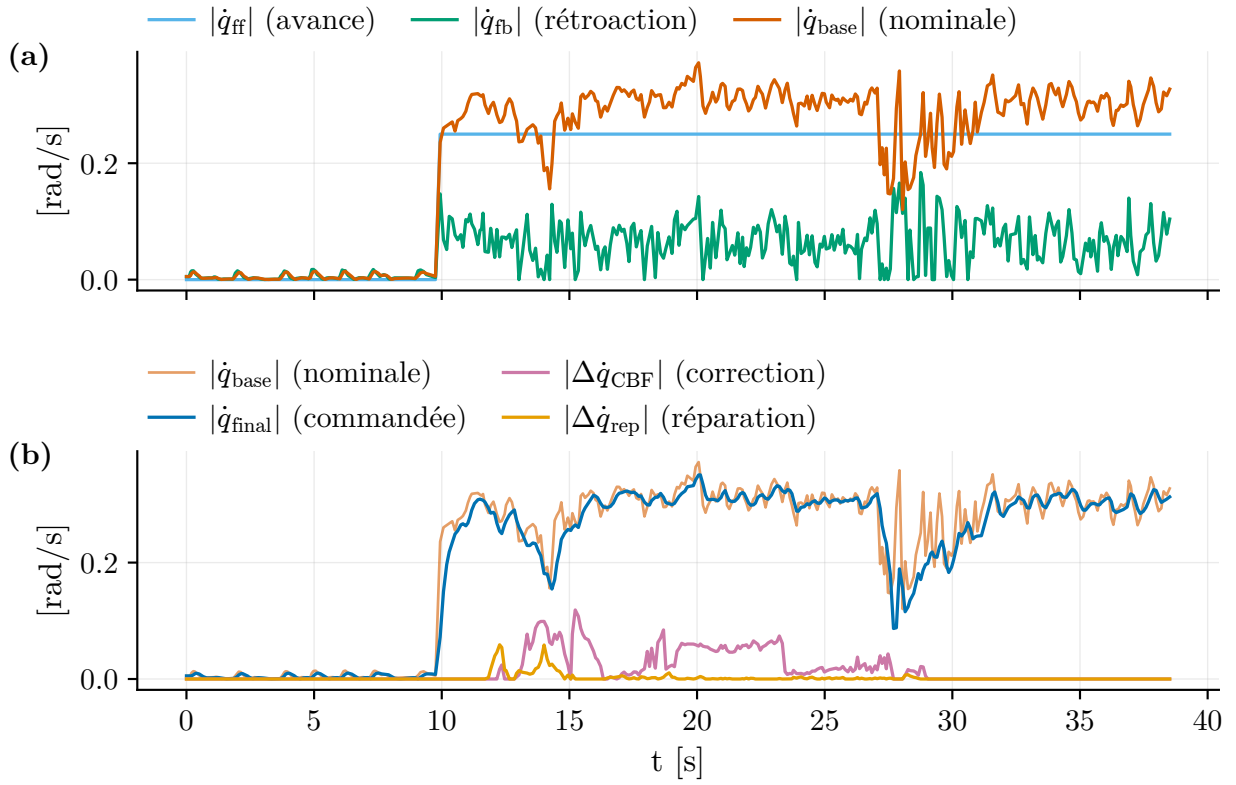


Figure 5.14 Décomposition des vitesses articulaires. (a) Construction de la vitesse nominale : feedforward retimé à norme constante $\dot{q}_{ff}$, correction de suivi $\dot{q}_{fb}$ et vitesse nominale filtrée $\dot{q}_{base}$. (b) Action du filtre de sécurité : corrections de projection $\Delta\dot{q}_{CBF}$ et de réparation $\Delta\dot{q}_{rep}$ (reconstituée par rejeu du filtre de sortie), et vitesse commandée $\dot{q}_{final}$. Courbes lissées sur 0,25 s.

la marge). L'écart entre les deux courbes se lit donc directement comme l'amplitude de la modification. Hors des zones d'activation, $\|\dot{\mathbf{q}}_{\text{final}}\|$ demeure légèrement sous $\|\dot{\mathbf{q}}_{\text{base}}\|$ ($\approx 5\%$ en moyenne) : ce n'est pas une intervention de sécurité mais l'effet du filtre passe-bas de sortie ($\tau_{\text{safe}} = 0,2$ s), qui, en moyennant des vecteurs dont la direction fluctue d'un cycle à l'autre, produit une norme inférieure à la moyenne des normes — le même mécanisme d'annulation vectorielle que celui décrit à la Section 5.5.3.

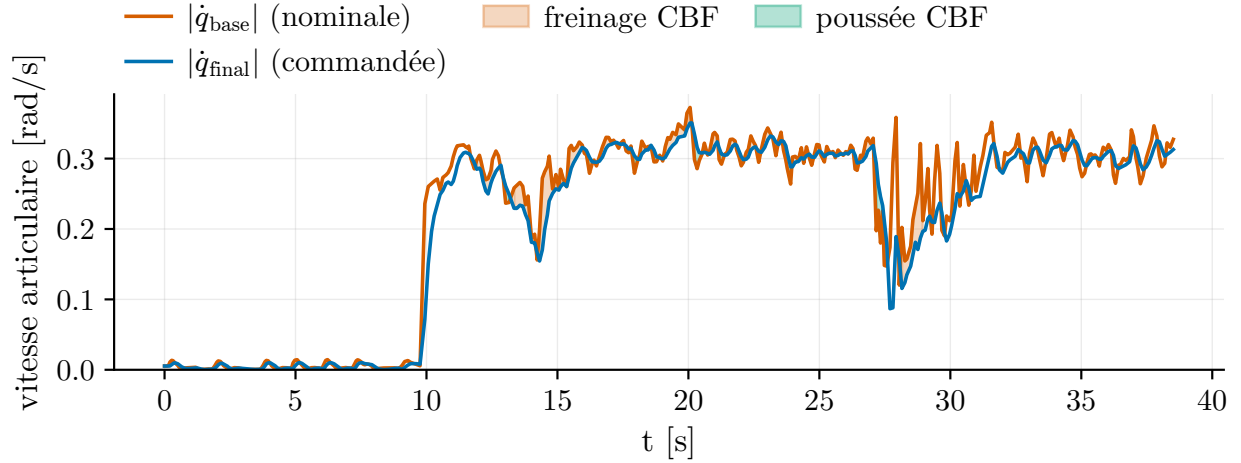


Figure 5.15 Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.

5.5.3 Effet du rééchantillonnage du planificateur sur la vitesse exécutée

La décomposition révèle un effet systémique du fonctionnement en horizon glissant. Bien que $\|\dot{\mathbf{q}}_{\text{ff}}\| = 0,25$ rad/s et que la correction de suivi soit en moyenne alignée avec le feed-forward (composante moyenne de $+0,03$ rad/s le long de $\dot{\mathbf{q}}_{\text{ff}}$), la vitesse nominale filtrée ne moyenne que $\approx 0,15$ rad/s. La cause n'est pas un freinage mais un phénomène vectoriel : à chaque régénération, la *direction* instantanée de $\dot{\mathbf{q}}_{\text{ff}}$ saute (rotations atteignant 80° entre cycles consécutifs au 95° centile), et le filtre passe-bas, en moyennant des vecteurs dont la direction alterne à 12 Hz, produit un vecteur de norme inférieure à la moyenne des normes. Près de 40 % de la vitesse demandée est ainsi perdue par annulation vectorielle, indépendamment de tout obstacle. Ce constat motive l'agrégation temporelle des trajectoires générées (moyennage pondéré des dernières générations) comme amélioration future du planificateur.

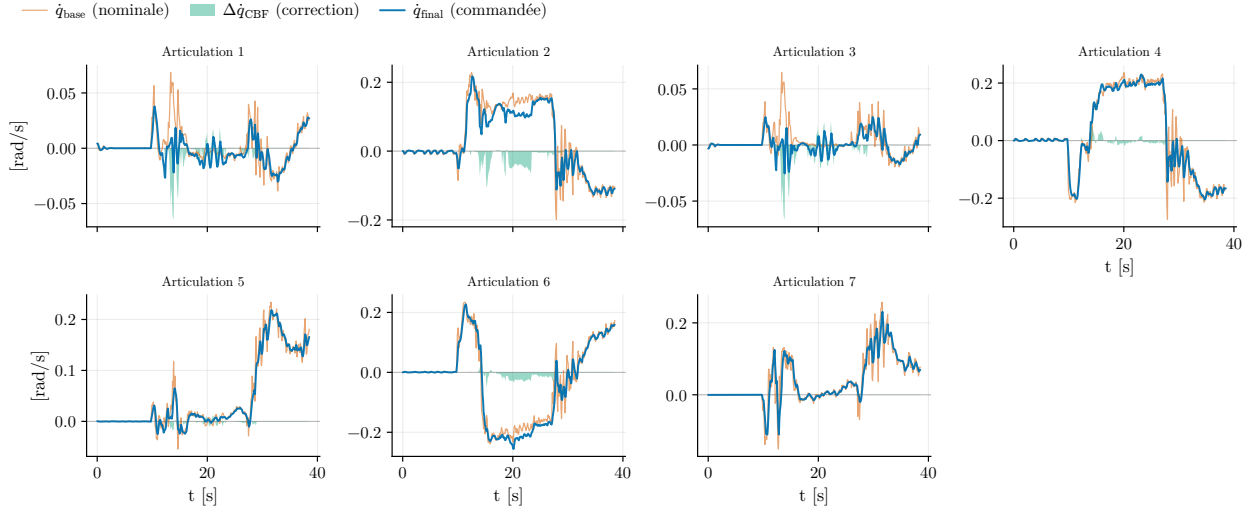


Figure 5.16 Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.

5.5.4 Effet du filtrage sur la commande sécurisée

La Figure 5.17 compare la norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie. Le filtre supprime la gigue de direction du gradient induite par le rafraîchissement de la sélection des points critiques à 30 Hz, au prix d'un retard de l'ordre de τ_{safe} ; la contrainte de sécurité est réévaluée après filtrage par l'étape de réparation (Section 4.7.5).

5.5.5 Analyse de l'interface de commande

L'instrumentation du rapport entre le taux de variation de la barrière commandé ($\nabla h^T \dot{\mathbf{q}}_{\text{final}}$) et celui réellement exécuté ($\nabla h^T \dot{\mathbf{q}}_{\text{mes}}$) a mis en évidence deux modes de défaillance de l'interface entre le filtre de sécurité et le contrôleur de position bas niveau, justifiant a posteriori les choix de conception de la Section 4.7.5. Avec une consigne ré-ancrée à chaque cycle sur la position mesurée, le rapport exécuté/commandé mesuré était proche de zéro : les corrections de sécurité étaient réabsorbées avant d'être réalisées, et le robot demeurerait immobilisé contre la frontière malgré une commande de dégagement soutenue. À l'inverse, avec un pas d'intégration supérieur à la période réelle de la boucle, la référence avançait plus vite que la vitesse certifiée et le rapport devenait négatif et supérieur à l'unité en valeur absolue : le robot traversait la frontière en oscillant, produisant des violations répétées de la marge. L'interface retenue — intégration de la référence persistante au pas réel Δt avec borne d'avance

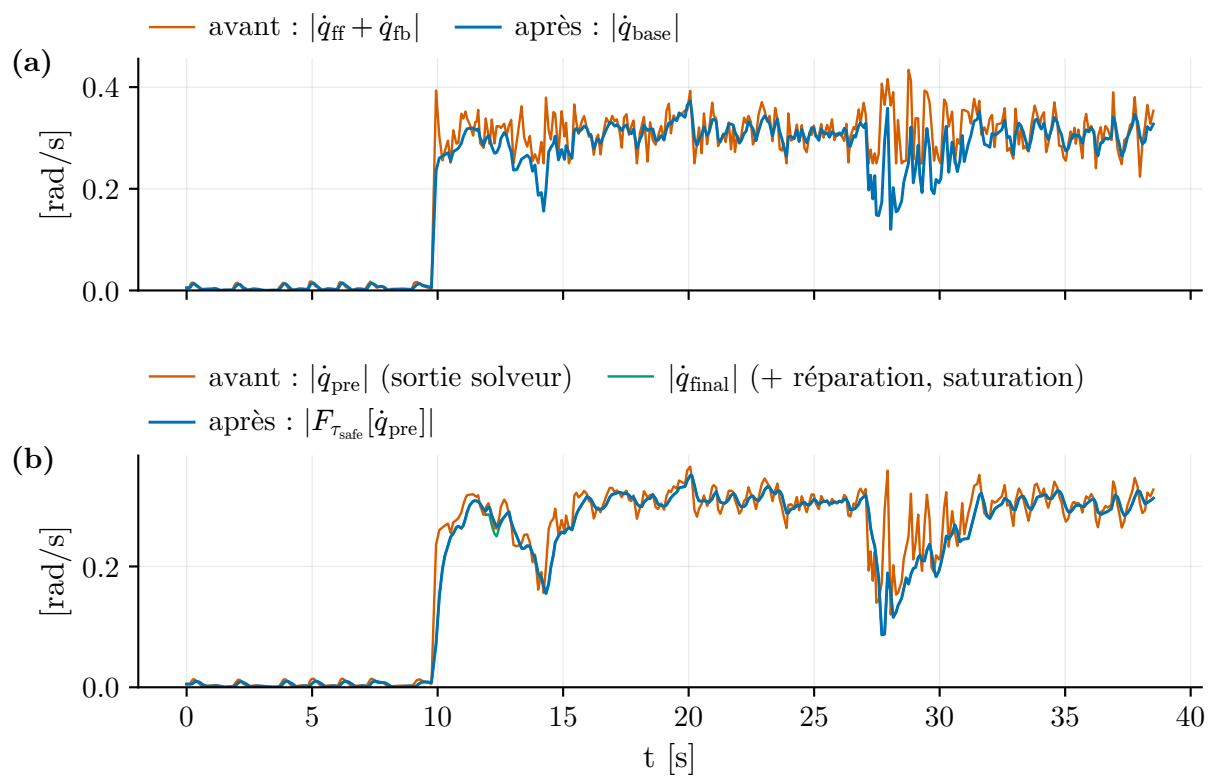


Figure 5.17 Norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie ($\tau_{safe} = 0,2$ s).

— élimine ces deux régimes.

5.6 Performance et temps de calcul

Tableau 5.1 Statistiques descriptives par étape (stage)

Stage	n	Mean	std/min	Med	p95	Max
cbf_command	1660	1.88	2.12	1.38	4.98	31.10
cbf_correction	1660	7.58	5.41	6.50	17.65	41.44
cbf_total	1660	9.91	6.43	8.60	22.06	57.81
critical_point_selection	832	42.27	18.35	39.62	75.49	148.81
fm_encode	263	1.51	2.21	1.02	2.85	20.87
fm_generation	263	62.07	6.40	61.62	73.89	83.89
ik	255	0.27	0.09	0.25	0.43	0.93
persistent_evict	709	3.17	1.74	2.76	5.90	22.08
persistent_integrate	1753	1.02	1.06	0.84	1.80	23.85
sam2_track	132	53.85	15.10	52.77	78.66	91.14
sam3_detect	2	182.45	152.72	182.45	319.89	335.16

À compléter.

5.7 Performances globales et limites

5.7.1 Taux de succès des scénarios

Pick and place

Robot assisted feeding

5.7.2 Cas d'échec

À compléter.

5.8 Discussion

CHAPITRE 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

RÉFÉRENCES

- [1] S. Karaman et E. Frazzoli, “Sampling-Based Algorithms for Optimal Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 30, n°. 7, p. 846–894, 2011.
- [2] I. A. Şucan, M. Moll et L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, n°. 4, p. 72–82, 2012.
- [3] S. Liu et P. Liu, “Robot Motion Planning Benchmarking and Optimization using Motion Planning Pipeline,” 2021, arXiv preprint.
- [4] M. Zucker *et al.*, “CHOMP : Covariant Hamiltonian Optimization for Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 32, n°. 9-10, p. 1164–1193, 2013.
- [5] J. Schulman *et al.*, “Motion Planning with Sequential Convex Optimization and Convex Collision Checking,” *The International Journal of Robotics Research (IJRR)*, vol. 33, n°. 9, p. 1251–1270, 2014.
- [6] J. J. Kuffner et S. M. LaValle, “RRT-Connect : An Efficient Randomized Spatio-temporal Planner,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, p. 995–1001.
- [7] L. E. Kavraki *et al.*, “Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, n°. 4, p. 566–580, 1996.
- [8] C. G. Atkeson et S. Schaal, “Robot Learning from Demonstration,” dans *International Conference on Machine Learning (ICML)*, 1997, p. 12–20.
- [9] S. Schaal, “Is Imitation Learning the Way to Eager Robotics?” *Trends in Cognitive Sciences*, vol. 3, n°. 6, p. 233–242, 1999.
- [10] B. D. Argall *et al.*, “A survey of robot learning from demonstration,” 2009, arXiv preprint.
- [11] D. A. Pomerleau, “ALVINN : An Autonomous Land Vehicle in a Neural Network,” dans *Advances in Neural Information Processing Systems (NeurIPS)*, 1989, p. 305–313.
- [12] M. Bain et C. Sammut, “A Framework for Behavioural Cloning,” dans *Machine Intelligence 15*, 1995, p. 103–129.
- [13] T. Z. Zhao *et al.*, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” 2023, arXiv preprint.

- [14] P. Sundaresan, S. Belkhale et D. Sadigh, “Learning Visuo-Haptic Skewering Strategies for Robot-Assisted Feeding,” 2022, arXiv preprint.
- [15] R. Liu, A. Bhaskar et P. Tokekar, “Adaptive Visual Imitation Learning for Robotic Assisted Feeding Across Varied Bowl Configurations and Food Types,” 2024, arXiv preprint.
- [16] P. Florence *et al.*, “Implicit Behavioral Cloning,” 2021, arXiv preprint.
- [17] D. Jarrett, I. Bica et M. v. d. Schaar, “Strictly Batch Imitation Learning by Energy-based Distribution Matching,” 2021, arXiv preprint.
- [18] P. Florence, L. Manuelli et R. Tedrake, “Self-Supervised Correspondence in Visuomotor Policy Learning,” 2019, arXiv preprint.
- [19] J. Luo *et al.*, “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” 2025, arXiv preprint.
- [20] A. Gupta *et al.*, “Relay Policy Learning : Solving Long-Horizon Tasks via Imitation and Reinforcement Learning,” 2019, arXiv preprint.
- [21] S. M. Khansari-Zadeh et A. Billard, “Learning Stable Nonlinear Dynamical Systems With Gaussian Mixture Models,” 2011, arXiv preprint.
- [22] R. Wolf *et al.*, “Diffusion Models for Robotic Manipulation : A Survey,” *arXiv preprint arXiv :2504.08438*, 2025.
- [23] J. Cho, M. Kim et S. Jung, “A Survey on Flow Matching for Robotic Trajectory Generation and Control,” 2025, arXiv preprint.
- [24] C. Chi *et al.*, “Diffusion Policy : Visuomotor Policy Learning via Action Diffusion,” dans *Proceedings of Robotics : Science and Systems (RSS)*, 2023.
- [25] J. Ho, A. Jain et P. Abbeel, “Denoising Diffusion Probabilistic Models,” 2020, arXiv preprint.
- [26] M. Janner *et al.*, “Planning with Diffusion for Flexible Behavior Synthesis,” 2022, arXiv preprint.
- [27] S. Huang *et al.*, “Diffusion-based Generation, Optimization, and Planning in 3D Scenes,” 2023, arXiv preprint.
- [28] Y. Ze *et al.*, “3D Diffusion Policy : Generalizable Visuomotor Policy Learning via Simple 3D Representations,” 2024, arXiv preprint.
- [29] G. Lu *et al.*, “ManiCM : Real-time 3D Diffusion Policy via Consistency Model for Robotic Manipulation,” 2025, arXiv preprint.
- [30] A. Prasad *et al.*, “Consistency Policy : Accelerated Visuomotor Policies via Consistency Distillation,” 2024, arXiv preprint.

- [31] Z. Wang *et al.*, “One-Step Diffusion Policy : Fast Visuomotor Policies via Diffusion Distillation,” 2024, arXiv preprint.
- [32] Y. Lipman *et al.*, “Flow Matching for Generative Modeling,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [33] A. Tong *et al.*, “Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [34] X. Liu, C. Gong et Q. Liu, “Flow Straight and Fast : Learning to Generate and Transfer Data with Rectified Flow,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [35] E. Chisari *et al.*, “Learning Robotic Manipulation Policies from Point Clouds with Conditional Flow Matching,” 2024, arXiv preprint.
- [36] G. Yan *et al.*, “ManiFlow : A General Robot Manipulation Policy via Consistency Flow Training,” 2025, arXiv preprint.
- [37] Z. Geng *et al.*, “Mean Flows for One-step Generative Modeling,” 2025, arXiv preprint.
- [38] H. Fang *et al.*, “OMP : One-step Meanflow Policy with Directional Alignment,” 2026, arXiv preprint.
- [39] S. Jiang *et al.*, “Streaming Flow Policy : Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories,” 2025, arXiv preprint.
- [40] K. Nguyen *et al.*, “FlowMP : Learning Motion Fields for Robot Planning with Conditional Flow Matching,” 2025, arXiv preprint.
- [41] D. Soleymanzadeh, X. Liang et M. Zheng, “Flow Motion Policy : Manipulator Motion Planning with Flow Matching Models,” 2026, arXiv preprint.
- [42] M. Braun *et al.*, “Riemannian Flow Matching Policy for Robot Motion Learning,” 2024, arXiv preprint.
- [43] A. Brohan *et al.*, “RT-2 : Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” 2023, arXiv preprint.
- [44] M. J. Kim *et al.*, “OpenVLA : An Open-Source Vision-Language-Action Model,” 2024, arXiv preprint.
- [45] R. Sapkota *et al.*, “Vision-Language-Action (VLA) Models : Concepts, Progress, Applications and Challenges,” 2026, arXiv preprint.
- [46] D. Li *et al.*, “What Foundation Models can Bring for Robot Learning in Manipulation : A Survey,” *The International Journal of Robotics Research*, 2025.

- [47] J. Wen *et al.*, “TinyVLA : Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation,” 2025, arXiv preprint.
- [48] S. Liu *et al.*, “Vision-language model-driven scene understanding and robotic object manipulation,” 2024, arXiv preprint.
- [49] O. Mees, L. Hermann et W. Burgard, “What Matters in Language Conditioned Robotic Imitation Learning over Unstructured Data,” 2022, arXiv preprint.
- [50] N. Ingelhart *et al.*, “A Robotic Skill Learning System Built Upon Diffusion Policies and Foundation Models,” 2024, arXiv preprint.
- [51] G. Yin *et al.*, “CodeDiffuser : Attention-Enhanced Diffusion Policy via VLM-Generated Code for Instruction Ambiguity,” 2025, arXiv preprint.
- [52] A. Kirillov *et al.*, “Segment Anything,” dans *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, p. 4015–4026.
- [53] Meta AI Research, “SAM 3 : Segment Anything with Concepts,” *arXiv preprint*, 2025.
- [54] N. Ravi *et al.*, “SAM 2 : Segment Anything in Images and Videos,” *arXiv preprint arXiv :2408.00714*, 2024.
- [55] D. Seita *et al.*, “ToolFlowNet : Robotic Manipulation with Tools via Predicting Tool Flow from Point Clouds,” 2022, arXiv preprint.
- [56] H. Li *et al.*, “Language-Guided Object-Centric Diffusion Policy for Generalizable and Collision-Aware Manipulation,” 2025, arXiv preprint.
- [57] R. Yang *et al.*, “FP3 : A 3D Foundation Policy for Robotic Manipulation,” 2025, arXiv preprint.
- [58] Y. Fu *et al.*, “CordViP : Correspondence-based Visuomotor Policy for Dexterous Manipulation in Real-World,” 2025, arXiv preprint.
- [59] R. Yu *et al.*, “No Need for Real 3D : Fusing 2D Vision with Pseudo 3D Representations for Robotic Manipulation Learning,” 2025, arXiv preprint.
- [60] Z. Ni *et al.*, “VO-DP : Semantic-Geometric Adaptive Diffusion Policy for Vision-Only Robotic Manipulation,” 2025, arXiv preprint.
- [61] J. Jones *et al.*, “Beyond Sight : Finetuning Generalist Robot Policies with Heterogeneous Sensors via Language Grounding,” 2025, arXiv preprint.
- [62] J. Morris *et al.*, “DynoSAM : Open-Source Smoothing and Mapping Framework for Dynamic SLAM,” 2025, arXiv preprint.
- [63] Y. Jin *et al.*, “SE(3)-PoseFlow : Estimating 6D Pose Distributions for Uncertainty-Aware Robotic Manipulation,” 2025, arXiv preprint.

- [64] A. Hornung *et al.*, “OctoMap : An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, n°. 3, p. 189–206, 2013.
- [65] H. Oleynikova *et al.*, “Voxblox : Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, p. 1366–1373.
- [66] A. Millane *et al.*, “nvblox : GPU-Accelerated Incremental Signed Distance Field Mapping,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [67] L. Han *et al.*, “FIESTA : Fast Incremental Euclidean Distance Fields for Online Motion Planning of Aerial Robots,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, p. 4423–4430.
- [68] Y. Pan *et al.*, “Voxfield : Non-Projective Signed Distance Fields for Online Planning and 3D Reconstruction,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, p. 10 173–10 180.
- [69] W. B. Bedada et G. Palli, “Real Time Collision Avoidance with GPU-Computed Distance Maps,” 2024, arXiv preprint.
- [70] J. Ortiz *et al.*, “iSDF : Real-Time Neural Signed Distance Fields for Robot Perception,” dans *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, p. 4654–4663.
- [71] E. Sucar *et al.*, “iMAP : Implicit Mapping and Positioning in Real-Time,” dans *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021, p. 6229–6238.
- [72] Z. Zhu *et al.*, “NICE-SLAM : Neural Implicit Scalable Encoding for SLAM,” dans *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, p. 12 776–12 786.
- [73] B. Sundaralingam *et al.*, “CuRobo : Parallelized Collision-Free Robot Motion Generation,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, p. 8112–8119.
- [74] J. Lee *et al.*, “Learning Fast, Tool aware Collision Avoidance for Collaborative Robots,” 2025, arXiv preprint.
- [75] Y. Li *et al.*, “Representing robot geometry as distance fields : Applications to whole-body manipulation,” dans *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024, p. 15 351–15 357.
- [76] B. Liu *et al.*, “Collision-free Motion Generation Based on Stochastic Optimization and Composite Signed Distance Field Networks of Articulated Robot,” 2023, arXiv preprint.

- [77] A. Govoni *et al.*, “Real-time collision avoidance with robot distance fields in a task-priority framework,” *Robotics and Autonomous Systems*, vol. 200, p. 105394, 2026.
- [78] O. Khatib, “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots,” *The International Journal of Robotics Research (IJRR)*, vol. 5, n^o. 1, p. 90–98, 1986.
- [79] H. Ding *et al.*, “Towards Safe Imitation Learning via Potential Field-Guided Flow Matching,” 2025, arXiv preprint.
- [80] A. D. Ames *et al.*, “Control Barrier Functions : Theory and Applications,” 2019, arXiv preprint.
- [81] P. Wieland et F. Allgöwer, “CONSTRUCTIVE SAFETY USING CONTROL BARRIER FUNCTIONS,” 2007, arXiv preprint.
- [82] A. D. Ames *et al.*, “Control Barrier Function Based Quadratic Programs for Safety Critical Systems,” 2017, arXiv preprint.
- [83] W. S. Cortez *et al.*, “Control Barrier Functions for Mechanical Systems : Theory and Application to Robotic Grasping,” 2019, arXiv preprint.
- [84] A. Agrawal et K. Sreenath, “Discrete Control Barrier Functions for Safety-Critical Control of Discrete Systems with Application to Bipedal Robot Navigation,” dans *Robotics : Science and Systems (RSS)*, 2017.
- [85] L. E. Beaver, “Optimal Control Barrier Functions : Maximizing the Action Space Subject to Control Bounds,” 2024, arXiv preprint.
- [86] X. Dai *et al.*, “SafeFlow : Safe Robot Motion Planning with Flow Matching via Control Barrier Functions,” 2025, arXiv preprint.
- [87] J. Yang, S. Jang et S. Han, “SafeFlowMatcher : Safe and Fast Planning using Flow Matching with Control Barrier Functions,” 2026, arXiv preprint.
- [88] J. Long *et al.*, “Constraining Streaming Flow Models for Adapting Learned Robot Trajectory Distributions,” 2026, arXiv preprint.
- [89] K. Mizuta et K. Leung, “CoBL-Diffusion : Diffusion-Based Conditional Robot Planning in Dynamic Environments Using Control Barrier and Lyapunov Functions,” 2024, arXiv preprint.
- [90] Y. Song *et al.*, “OmniGuide : Universal Guidance Fields for Enhancing Generalist Robot Policies,” 2026, arXiv preprint.
- [91] Z. Yang *et al.*, “UniConFlow : A Unified Constrained Flow-Matching Framework for Certified Motion Planning,” 2026, arXiv preprint.
- [92] T.-Y. Huang *et al.*, “SAD-Flower : Flow Matching for Safe, Admissible, and Dynamically Consistent Planning,” 2026, arXiv preprint.

- [93] K. Mizuta et K. Leung, “Unified Generation-Refinement Planning : Bridging Guided Flow Matching and Sampling-Based MPC for Social Navigation,” 2026, arXiv preprint.
- [94] R. Römer *et al.*, “From Demonstrations to Safe Deployment : Path-Consistent Safety Filtering for Diffusion Policies,” 2026, arXiv preprint.
- [95] E. Welte *et al.*, “FlowCorrect : Efficient Interactive Correction of Generative Flow Policies for Robotic Manipulation,” 2026, arXiv preprint.
- [96] A. Liao *et al.*, “Delay-Aware Diffusion Policy : Bridging the Observation-Execution Gap in Dynamic Tasks,” 2025, arXiv preprint.
- [97] X. Ye *et al.*, “RA-DP : Rapid Adaptive Diffusion Policy for Training-Free High-frequency Robotics Replanning,” 2025, arXiv preprint.
- [98] F. Xiang *et al.*, “SAPIEN : A simulated part-based interactive environment,” dans *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [99] Y. Nakamura et H. Hanafusa, “Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 108, n°. 3, p. 163–171, 1986.
- [100] C. W. Wampler, “Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, n°. 1, p. 93–101, 1986.
- [101] A. Li’geois, “Automatic Supervisory Control of the Configuration and Behavior of Multibody Mechanisms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 7, n°. 12, p. 868–871, 1977.

ANNEXE A DÉMO

Texte de l'annexe A. Remarquez que la phrase précédente se termine par une lettre majuscule suivie d'un point. On indique explicitement cette situation à $\text{\LaTeX}$ afin que ce dernier ajuste correctement l'espacement entre le point final de la phrase et le début de la phrase suivante.

ANNEXE B ENCORE UNE ANNEXE / ANOTHER APPENDIX

Texte de l'annexe B en mode «landscape».

ANNEXE C UNE DERNIÈRE ANNEXE / THE LAST APPENDIX

Texte de l'annexe C.